

A PROPOSED DESIGN AND IMPLEMENTATION

DSN4091 – CAPSTONE PROJECT PHASE-I

Phase – I Report

Submitted by

KISLAY ANAND (23BAI10359)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



VIT[®]
BHOPAL
www.vitbhopal.ac.in

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY, SEHORE,

MADHYA PRADESH – 466114

September 2026



BONAFIDE CERTIFICATE

Certified that this project report titled “**A PROPOSED DESIGN AND IMPLEMENTATION FOR AUTOINSPECT-X: CNN-TRANSFORMER HYBRID DEEP LEARNING FOR VEHICLE DAMAGE SEGMENTATION**” is the bonafide work of “**KISLAY ANAND (23BAI10359)**” who carried out the project work (DSN4091 – Capstone Project Phase-I) under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR

Dr. Pradeep Kumar Mishra

Senior Assistant Professor (Gr-2)

School of Computing Science Engineering and
Artificial Intelligence

VIT BHOPAL UNIVERSITY

PROJECT GUIDE

Dr. Rudra Kalyan Nayak

Senior Assistant Professor (Gr-2)

School of Computing Science Engineering and
Artificial Intelligence

VIT BHOPAL UNIVERSITY

The DSN4091 – Capstone Project Phase-I Viva Voce Examination is held on _____

ACKNOWLEDGEMENT

First and foremost, I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I would like to thank my internal guide Dr. Rudra Kalyan Nayak, for continually guiding and actively participating in my project, and giving valuable suggestions to complete the project work.

I wish to express my heartfelt gratitude to Dr. Pradeep Kumar Mishra, PC-Lead, School of Computing Science Engineering and Artificial Intelligence, for much of his valuable support and encouragement in carrying out this work.

I wish to express my heartfelt gratitude to Dr. Pon Harshavardhanan, Dean, School of Computing Science Engineering and Artificial Intelligence, for much of his valuable support and encouragement in carrying out this work.

I would like to thank all the technical and teaching staff of the School of Computing Science Engineering and Artificial Intelligence, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

ABSTRACT

Vehicle-exterior damage inspection is still largely a manual, judgement-based activity. Insurance claims, rental-car returns and garage intake rely on a human looking at photographs and describing what is visible; many automated tools overreach by presenting uncalibrated model output as measured ground truth, or by inventing repair-cost figures that cannot be derived from an uncontrolled photograph. This project develops AutoInspect-X, a photo-first vehicle damage inspection system that keeps model output honest. A single photograph of the damaged area is first validated by a capture-quality gate (luminance, glare, blur and contrast heuristics); when it passes, a deep segmentation network predicts per-pixel damage classes; and an evidence module derives transparent, image-denominator area ratios and confidence values that are explicitly labelled as model predictions, never as verified physical measurements. The work is grounded in the CarDD dataset (4,000 images with official train/validation/test splits), and compares a CNN+Transformer hybrid segmentation model (ResNet-34 encoder, bottleneck transformer and U-Net-style decoder, deep-supervised Dice+Focal training) against a ResNet-34 U-Net baseline. The system itself comprises a FastAPI backend, a LangChain/LangGraph conversational assistant (Groq-backed online, deterministic offline), an opt-in consent pipeline that stores training samples only with explicit user agreement, and a Next.js/React demonstration front end. This Phase-I report records the analysis: problem definition, literature study, requirement artefacts, design methodology, implementation, measured controlled results, a walk-through of validation scenarios, and the limitations and future scope. Repair costs and pricing are explicitly out of scope. The controlled catalogue holds two architectures at two seeds each: best validation foreground mIoU is 0.667659 and 0.668115 for the baseline (seeds 42, 1337) and 0.670167 and 0.673167 for the hybrid, with product default final60_hybrid_seed42. The two-seed difference is recorded as a small, unreplicated margin, not as a final scientific conclusion (ADR 0012, ADR 0013).

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Fig. 1.1	Photo-first inspection workflow of AutoInspect-X.	1
Fig. 4.1	Software architecture of the AutoInspect-X demo system.	11
Fig. 5.1	First frame of the scroll-driven cinematic home-page intro (frame 1/757).	18
Fig. 5.2	Near-end frame of the cinematic intro sequence (frame 757/757).	18
Fig. 5.3	Inspection composer at /demo with upload prompt and free-text field.	19
Fig. 5.4	Blurred input photograph used to exercise the capture-quality gate.	20
Fig. 5.5	Capture-quality rejection (TOO_BLURRY) with retake guidance; no mask is produced.	21
Fig. 5.6	Minor-scratch test photograph (street scene).	22
Fig. 5.7	Minor-scratch result - detected region, mean confidence, image-area fraction, consent prompt.	23
Fig. 5.8	Minor dent/scratch test photograph (close-up).	24
Fig. 5.9	Minor-damage result - scratch chip with confidence, damage-area ratio, consent granted.	25
Fig. 5.10	Medium-damage test photograph.	26
Fig. 5.11	Medium-damage result - dent, scratch and lamp-broken chips with consent prompt.	27
Fig. 5.12	Major-damage test photograph.	27
Fig. 5.13	Major-damage result - dent with a small glass-shatter region and consent prompt.	28
Fig. 5.14	Multi-cue (confusing) test photograph.	29
Fig. 5.15	Multi-cue result - dent, crack and tire-flat chips with mean confidence.	30
Fig. A.1	Composer with the photo attached before analysis (minor-scratch session).	37
Fig. A.2	Assistant narrative for a minor dent (the assistant's "Here is	38

	what the visual model identified” summary).	
Fig. A.3	Assistant narrative for a medium-damage photograph.	39
Fig. A.4	Major-damage result with consent granted and re-analysis confirmation.	40
Fig. A.5	Assistant narrative for the multi-cue (confusing) photograph.	41

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	v
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
CHAPTER 1	PROJECT DESCRIPTION AND OUTLINE	1
	1.1 INTRODUCTION	1
	1.2 MOTIVATION FOR THE WORK	1
	1.3 PROBLEM STATEMENT	2
	1.4 OBJECTIVE OF THE WORK	2
	1.5 SUMMARY	3
CHAPTER 2	RELATED WORK INVESTIGATION	4
	2.1 EXISTING APPROACHES/METHODS	4
	2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS	4
	2.3 SUMMARY	5
CHAPTER 3	REQUIREMENT ARTIFACTS	6
	3.1 INTRODUCTION	6
	3.2 HARDWARE AND SOFTWARE REQUIREMENTS	6
	3.3 SPECIFIC PROJECT REQUIREMENTS	7
	3.3.1 DATA REQUIREMENT	7
	3.3.2 FUNCTIONS REQUIREMENT	7
	3.3.3 PERFORMANCE AND SECURITY REQUIREMENTS	8
	3.3.4 LOOK AND FEEL REQUIREMENTS	8
	3.4 SUMMARY	9
CHAPTER 4	DESIGN METHODOLOGY AND ITS NOVELTY	10
	4.1 METHODOLOGY AND GOAL	10
	4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS	10
	4.3 SOFTWARE AND SYSTEM ARCHITECTURE	11
	4.4 DATA FLOW / PROCESSING PIPELINE	13
	4.5 ML / CV METHODOLOGY	14
	4.6 USER INTERFACE AND INTERACTION DESIGN	15
	4.7 NOVELTY	15
	4.8 SUMMARY	16
CHAPTER 5	TECHNICAL IMPLEMENTATION AND ANALYSIS	17
	5.1 OUTLINE	17
	5.2 TECHNICAL CODING AND CODE SOLUTIONS	17
	5.3 PROTOTYPE SUBMISSION	17
	5.4 SUMMARY	31
CHAPTER 6	PROJECT OUTCOME AND APPLICABILITY	32
	6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM	32
	6.2 SIGNIFICANT PROJECT OUTCOMES	32
	6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS	34
	6.4 INFERENCE	34
CHAPTER 7	CONCLUSIONS AND RECOMMENDATION	35
	7.1 OUTLINE	35
	7.2 LIMITATION/CONSTRAINTS OF THE SYSTEM	35
	7.3 FUTURE ENHANCEMENTS	35
	7.4 INFERENCE	36
	APPENDIX A SCREENSHOTS	37

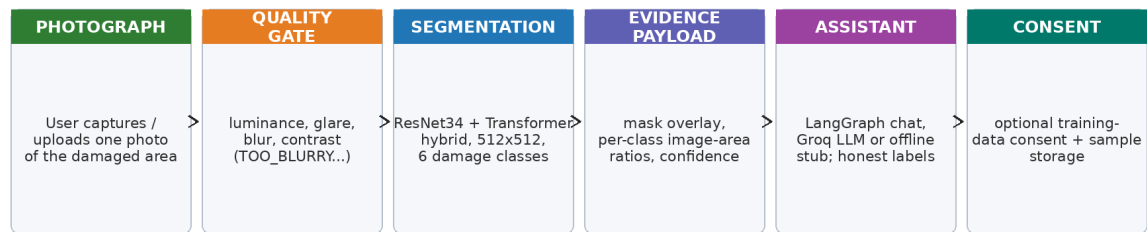
	APPENDIX B CODE LISTINGS	42
	REFERENCES	74

CHAPTER 1 PROJECT DESCRIPTION AND OUTLINE

1.1 INTRODUCTION

Vehicle damage claims, rental-car returns and garage intake all begin the same way: someone takes photographs and someone else reads them. The photographs are uncontrolled – different distances, angles, lighting and compression – and the reading is manual and subjective. Automated damage-inspection products promise to fix the second problem but usually inherit the first one, and many present model output as if it were a measured fact, occasionally adding confident-looking cost figures that no photograph can justify.

AutoInspect-X takes a deliberately photo-first and evidence-labelled approach. The system accepts one photograph of a damaged area, rejects photograph-quality problems that would make any interpretation unreliable (dark, glaring, blurry or low-contrast shots), and only then runs a deep segmentation model that predicts per-pixel damage classes. From that mask the system derives image-denominator area ratios and confidence values that are always labelled as model predictions. A conversational assistant explains the findings from the structured evidence and refuses to invent severity, repair actions or costs. If the user agrees, the anonymised sample can be kept for future training.



Damage-area values are DERIVED FEATURES (image-denominator ratios), never verified physical area.
Every finding is labelled MODEL PREDICTION with channel-wise confidence; a failed quality gate returns no mask.

Fig. 1.1 Photo-first inspection workflow of AutoInspect-X.

Figure 1.1 summarises the workflow: capture, quality gate, segmentation, evidence payload, assistant and optional consent. The design rests on three integrity rules carried throughout the repository: real ground truth, synthetic labels, derived features and model predictions are kept in distinct labelled categories; area ratios are normalised by the image, never claimed as physical cm²; and any cost or repair prediction is out of scope (recorded in the project's decision log, ADR 0011).

1.2 MOTIVATION FOR THE WORK

The motivation for this work comes from three observed gaps in how vehicle damage is handled today. First, inspection is manual: a claims adjuster or a workshop mechanic spends time looking at photos that are often of poor quality, and the resulting written report is hard to reproduce. Second, automation has an honesty problem: off-the-shelf ‘damage detectors’ routinely quote damage percentages or repair costs that they cannot actually measure, and even academic tools rarely separate what a model predicted from what a human verified. Third, the data that would let a model improve is locked in customer photos, and collecting it without making the user explicitly consent is both legally fragile and ethically questionable.

A photo-first system addresses all three. A quality gate removes the inputs that make any automated reading unreliable before the model is ever invoked. Honest labels – image-denominator area, channel-wise confidence, low-confidence flags and model notes – let a human interpreter decide how much to trust the output. And an explicit consent pipeline turns everyday inspections, with the owner’s permission, into labelled training data. This project is therefore as much about the conversational and consent infrastructure around a segmentation model as about the model itself.

1.3 PROBLEM STATEMENT

Manual vehicle-damage documentation from photographs is time-consuming, subjective and hard to scale, while automated alternatives frequently present uncalibrated predictions – including fabricated cost figures – as ground truth, and typically ignore that many damaged-car photographs are too dark, blurry or glaring to be interpreted reliably at all. Formally, the problem addressed here is: given a single uncontrolled photograph of a damaged vehicle exterior, how can a system (i) decide whether the photograph is usable at all, (ii) produce a per-class damage segmentation over the CarDD damage categories, (iii) report the result as transparent, evidence-labelled features rather than as verified measurements, and (iv) explain the finding conversationally while refusing to fabricate severity, repairs or costs, and without using any user photograph without explicit consent?

A further subtlety is that legitimate damage-area ground truth in cm^2 cannot be measured from an uncontrolled photo without geometric calibration. The system therefore reports damage-area as a fraction of the image, which is a derived feature and is communicated as such. This keeps the science honest while still giving an adjuster a reproducible number.

1.4 OBJECTIVE OF THE WORK

The aim of the project is to design, implement and demonstrate a photo-first vehicle damage inspection system with evidence-labelled confidence. The specific objectives are:

1. Acquire and audit the CarDD-COCO dataset, preserve its official train/validation/test splits, and document its class and split statistics.
2. Design and implement two segmentation architectures: a ResNet-34 U-Net baseline and a CNN+Transformer hybrid (bottleneck transformer with additive fusion).
3. Implement a production-grade training harness (deep-supervised Dice+Focal loss, class-aware sampling, EMA, cosine schedule, mixed precision) and record reproducible controlled runs under a two-seed contract for each architecture.
4. Build a capture-quality gate using luminance, glare, blur and contrast heuristics so that unusable photographs are rejected before model inference, with actionable retake guidance.
5. Extract instance-level damage features from the predicted mask (connected components, image-denominator area ratios and confidence), always labelled as model predictions.
6. Implement the product layer around the model: a FastAPI REST backend, a LangChain/LangGraph conversational assistant (Groq-backed online, deterministic offline stub), a Next.js/React demonstration front end, and SQLite storage.
7. Provide an explicit, optional training-data consent flow that stores anonymised samples only when the user agrees.
8. Keep quality gates green (ruff, mypy strict, pytest, Playwright) and record limitations honestly in every model reply.

1.5 SUMMARY

This chapter introduced the photo-first philosophy, the motivation, and the problem and objectives that structure the rest of the report. Chapter 2 reviews datasets and segmentation approaches relevant to vehicle damage inspection. Chapter 3 collects the requirement artefacts. Chapter 4 presents the design methodology, architecture, models and the integrity rules. Chapter 5 documents the technical implementation with real code excerpts and the demonstration walk-through. Chapter 6 reports the controlled outcomes and applicability, and Chapter 7 concludes with limitations and future work.

CHAPTER 2 RELATED WORK INVESTIGATION

2.1 EXISTING APPROACHES/METHODS

The literature splits naturally into datasets, segmentation architectures and whole damage-detection systems. On the dataset side, CarDD is the key public resource: 4,000 vehicle-damage images with more than 9,000 surface-damage instances across dent, scratch, crack, glass shatter, lamp broken, and tire flat categories, distributed with official train/validation/test splits [Wang et al., IEEE TITS 2023]. VehiDE provides a second, differently collected damage set used here as a descriptive spectrum for training-time mixing, and CrashCar101 contributes curated real-world vehicle photos used as visual testing exemplars; neither supplies validated physical area measurements [Huynh et al., KSE 2023; Parsløv et al., WACV 2024].

On the architecture side, U-Net established the encoder–decoder-with-skip segmentation pattern that remains effective with small datasets [Ronneberger et al., 2015], while ResNet provides a strong, image-pretrained feature backbone [He et al., 2016]. Vision transformers brought explicit global context that convolutional receptive fields lack but are known to be data-hungry [Dosovitskiy et al., 2021]. Damage-specific hybrid work – for example CDA-Net, which couples attention with depth cues – shows that hybrid designs help for vehicles, though it depends on additional sensor input [Bhogale et al., 2023]. Instance imbalance in damage data is typically addressed with Focal loss [Lin et al., 2017], which this project combines with a soft-Dice objective.

On the system side, the literature is dominated by classification of damage type and rough severity from a single network, with almost no work exposing (i) a capture-quality gate, (ii) explicitly image-denominator area with a stated non-physicality caveat, or (iii) a conversational, evidence-grounded assistant. LLM chat assistants applied to inspection are promising but hallucinate unless their prompt is grounded in structured evidence and forbids repair/cost invention – exactly the contract implemented by the assistant service in this project.

2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS

Table 2.1 summarises the strengths and limitations of the approaches reviewed.

Approach / work	Strengths	Limitations
CarDD dataset (2023)	Large, dedicated vehicle-damage dataset; COCO instance masks;	Uncontrolled photos; no validated physical-area annotations; near-

	damage-type taxonomy matches this project.	duplicate images need leakage care.
VehiDE (2023); CrashCar101 (2024)	Additional damage spectra; useful for training-time mixing and visual exemplars.	Different taxonomies; neither validates cm ² ground truth.
U-Net (2015)	Strong with limited data; elegant skip connections.	Local receptive field; limited global context.
ResNet backbone (2016)	Robust image-pretrained features; strong transfer.	Fixed receptive field without attention; heavy for small VRAM devices.
Vision Transformer (2021)	Explicit global context.	Data-hungry; less sample-efficient on small damage sets.
CDA-Net (2023)	Hybrid attention + depth improves car damage cues.	Requires depth, unavailable for single consumer photos.
Focal loss (2017)	Targets class imbalance (imbalance is severe in CarDD).	Needs tuning with Dice; gamma/weight interaction.
Rule-based cost estimation	Cheap, plausible-looking output.	Not repair ground truth; cannot be derived from one photo; removed from scope (ADR 0011).
LLM chat assistants	Natural conversational explanations.	Hallucination risk unless grounded strictly in evidence; must forbid cost invention.

2.3 SUMMARY

The review justifies the design choices in this project: CarDD for a well-defined six-class damage taxonomy with official splits; a U-Net-style decoder on a ResNet-34 backbone for the baseline; a bottleneck transformer fused additively for global context; Focal+Dice training under class-aware sampling; and a strictly evidence-grounded assistant so that conversational convenience never becomes fabrication.

CHAPTER 3 REQUIREMENT ARTIFACTS

3.1 INTRODUCTION

This chapter collects the functional, data, performance, security and look-and-feel requirements distilled from the problem statement and the objectives, together with the hardware and software environment in which the system has been developed and can run.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.1 lists the reference hardware used for development and the minimum configuration needed to run the demonstration locally.

Component	Reference (used)	Minimum
Processor	Intel Core i5-12500H	Any x86-64 with 4+ cores
Memory	16 GB DDR4	8 GB
GPU	NVIDIA RTX 3050 Laptop, 4 GB VRAM (CUDA 12.1)	CPU-only works, slower
Storage	SSD (CarDD images $\approx$ 20 GB + checkpoints)	$\geq$ 40 GB free
Camera	any phone/DSLR (demo photographs)	any

Table 3.2 summarises the software toolchain, all open source.

The following table lists the precise technology stack. Every version is read from a repository manifest: backend versions are the CPU-runtime pins in requirements-ci.txt (the workstation runs the same minor versions as CUDA builds in the conda "ai" environment), frontend versions from package.json and package-lock.json, and the Python quality-gate constraints from pyproject.toml and uv.lock. Nothing is memorised.

Layer	Technology	Version (repo manifest)	Purpose	Evidence
Runtime language	Python	$\geq$ 3.11	API, ML and test code	pyproject.toml (requires-python)
Environment / packaging	uv, conda 'ai' env	uv 0.11.11	lock and run Python dependencies	uv.lock; CLAUDE.md tooling
Deep learning	PyTorch + torchvision	2.5.1 (+cu121 workstation / +cpu CI), 0.20.1	CNN + Transformer training and inference	requirements-ci.txt
Computer vision	OpenCV, NumPy, Pillow	4.13.0.92, 2.4.3, 12.1.1	image decode, connected components, overlays	requirements-ci.txt
Backend	FastAPI	0.136.1	REST API,	requirements-ci.txt

framework			validation, docs	
ASGI server	uvicorn	0.46.0	application server	requirements-ci.txt
Config / schemas	pydantic, pydantic-settings	2.12.5, 2.10.1	typed settings and response schemas	requirements-ci.txt
Agent stack	LangChain, LangGraph, langchain-groq, groq	1.2.15, 1.1.9, 1.1.2, 0.37.1	evidence-grounded conversational assistant	requirements-ci.txt
Multipart / HTTP	python-multipart, httpx	0.0.27, 0.28.1	file uploads and test/CI HTTP client	requirements-ci.txt
Persistence	SQLite (Python stdlib)	stdlib (3.11+)	sessions, image assets, consent, training samples	apps/api/storage/database.py
Frontend	Next.js, React	16.3.4, 19.2.8	web demo and cinematic intro UI	apps/web/package.json + package-lock.json
Styling	Tailwind CSS, PostCSS	4.3.3, 8.5.28	styling	apps/web/package-lock.json
Browser tests	Playwright	1.63.0	desktop / tablet / mobile end-to-end suites	apps/web/package-lock.json
Frontend tooling	TypeScript, ESLint, eslint-config-next	5.9.3, 9.39.5, 16.3.4	static typing and linting	apps/web/package-lock.json
Python QA / CI	pytest, ruff, mypy	pytest >= 8.4, ruff >= 0.14, mypy >= 1.18	unit tests; formatting/lint/types gates	pyproject.toml, uv.lock
LLM provider	Groq (ChatGroq; openai/gpt-oss-20b default)	groq 0.37.1; GROQ_MODEL	evidence-grounded chat; offline StubAssistant fallback	requirements-ci.txt; apps/api/agent/assistant.py

3.3 SPECIFIC PROJECT REQUIREMENTS

3.3.1 DATA REQUIREMENT

CarDD-COCO is used with its official splits preserved and measured locally on disk: the train split holds 2,816 images with 6,211 annotations, the validation split 810 images with 1,744 annotations, and the test split 374 images with 785 annotations (4,000 images in total). All four thousand images carry annotations. The train-split class distribution is heavily imbalanced – scratch (2,560 instances), dent (1,806), crack (651), lamp broken (494), glass shatter (475) and tire flat (225) – which drives the class-aware sampling and focal weighting. Split-scoped dataset adapters guarantee train/validation/test are never mixed, and image-level near-duplicates are documented in the repository audit so that leakage can be avoided.

3.3.2 FUNCTIONS REQUIREMENT

Session lifecycle: create an inspection session, upload one photo, analyse it, query its state, and delete it.

Capture-quality gating: reject dark, glaring, blurry or low-contrast photos with retake guidance before inference.

Damage segmentation: per-pixel prediction of the six CarDD damage classes plus background.

Evidence payload: predicted mask overlay, per-class and overall image-denominator areas, per-instance confidence.

Conversational explanation: the assistant explains findings from evidence and answers follow-ups; no cost/repair output.

Training-data consent: record GRANTED/DECLINED; store an anonymised sample only when granted (MODEL_SUGGESTED provenance).

Ephemeral cleanup: session photos and state are deleted when the session closes or expires; consented samples live separately.

3.3.3 PERFORMANCE AND SECURITY REQUIREMENTS

Performance: 512×512 inference on the 4 GB RTX 3050 (mixed precision) in about a second of black-box time; asynchronous chat turns.

Accuracy gate: the assistant repeats only structured evidence and flags low-confidence results as preliminary; every reply carries model notes.

Security: session and asset ids are whitelisted (never used unsanitised in paths); no secret keys are committed; the Groq key lives only in the local environment file, while CI runs the offline stub.

Data integrity: ground-truth categories, synthetic labels, derived features and predictions are kept distinct in code, storage and UI; area ratios are image-denominator only.

Privacy: chat history and personal identifiers are never stored for training; consented samples isolate the training directory from session cleanup.

3.3.4 LOOK AND FEEL REQUIREMENTS

The demonstration front end presents a dark, cinematic brand (a scroll-driven intro rendered from a 757-frame 30 FPS sequence) that transitions into a light, conversational inspector at /demo. The composer accepts a photo (drag, tap or capture) and a free-text message, then renders the assistant reply with model chips (damage classes), confidence, image-area fraction and an explicit “Demonstration build. Findings are machine predictions with explicit

labels – never verified damage” disclaimer under every result. Consent is a clearly optional step. Responsiveness is validated with Playwright over desktop, tablet and mobile viewports.

3.4 SUMMARY

Together these requirements define a minimal, honest, photo-first system: a session API around a quality gate, a segmentation engine, an evidence pipeline, an assistant, and an opt-in consent store, with a demonstration UI and CI quality gates.

CHAPTER 4 DESIGN METHODOLOGY AND ITS NOVELTY

4.1 METHODOLOGY AND GOAL

The project follows a phased, task-oriented methodology. Research phases 0–12 covered environment discovery, dataset audit and preparation, model framework selection, a smoke trainer, a full training harness, locked experiments, error analysis and the eventual architecture-specification revision. Product phases A–R covered, in order, repository audit, backend foundations, storage, the inference core, features and quality, the API wiring, the demonstration front end, the assistant/consent pipeline, end-to-end and real-engine journeys, Playwright suites, continuous integration and documentation. Each phase lands on a green quality gate (ruff, mypy strict, pytest), and material scoping changes are recorded as decision records – the most relevant being ADR 0010 (hybrid model lineage) and ADR 0011 (photo-first scope; cost, repair and questionnaire features removed).

The methodological goal is a system that is honest by construction: every stage carries labels that prevent model output from masquerading as ground truth, and the measurement conventions (image-denominator area, explicit preliminary flags) are enforced by tests.

4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS

Capture-quality module: computes sharpness (variance of Laplacian), mean luminance, bright fraction and contrast standard-deviation; maps them to VALID/TOO_DARK/EXCESSIVE_GLARE/TOO_BLURRY/INSUFFICIENT_CONTEXT; produces retake guidance, and never lets a rejected photo reach the model.

Segmentation engine: loads a checkpoint through a strict artefact contract (model_arch key, state dict), runs the training-time preprocess (512×512, 1/255, CHW), predicts 7-channel logits, and normalises deep-supervision outputs to the main head.

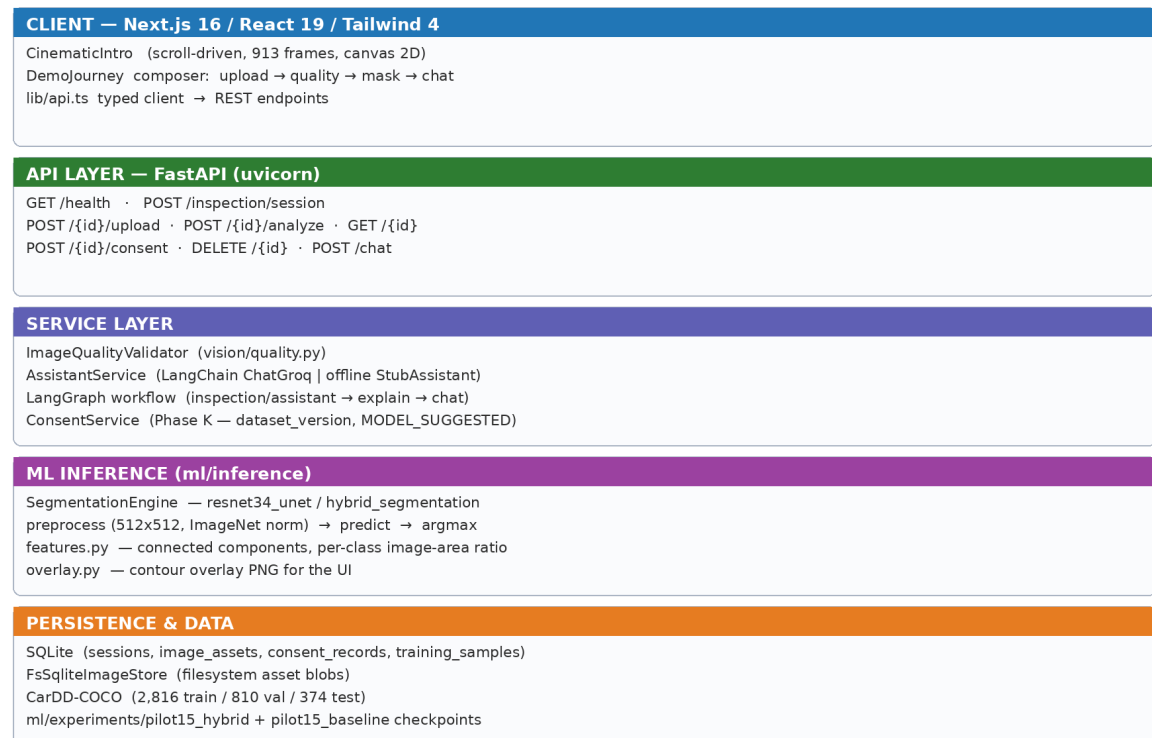
Feature extraction: runs connected components per damage class over the argmax mask, filters tiny components, and computes image-denominator area ratios, bounding boxes, centroids and per-instance confidence.

Assistant service: a protocol seam between the routers and either LangChain ChatGroq or the deterministic StubAssistant; grounded strictly in the prompt-safe evidence payload; forbids severity/cost invention.

Storage & consent service: SQLite session/store tables plus a filesystem image store; consent decisions recorded separately; training samples written with dataset version and MODEL_SUGGESTED provenance only on GRANTED.

4.3 SOFTWARE AND SYSTEM ARCHITECTURE

Figure 4.1 shows the layered architecture. The Next.js client talks only to the FastAPI router set (/health, /inspection/*, /chat). Services (quality validator, assistant, LangGraph workflow, consent) sit behind the routers, the segmentation and feature code is isolated under ml/inference, and persistence is split between SQLite and the filesystem image store. The ML training code (ml/training, ml/models) is deliberately not importable from production inference (enforced and tested), so the demo cannot accidentally retrain or use training-only transformations.



Arrows represent request/response flow: page → REST → services → ML inference → storage (SQLite + filesystem).

Fig. 4.1 Software architecture of the AutoInspect-X demo system.

The REST contract is summarised in Table 4.1.

Method & path	Behaviour
GET /health	liveness + model-version metadata
POST /inspection/session	create inspection session (uuid)
POST /inspection/{id}/upload	store one photo as an image asset

POST /inspection/{id}/analyze	quality gate; segmentation; evidence; assistant reply
GET /inspection/{id}	current state (messages, quality, inspection)
POST /inspection/{id}/consent	record GRANTED/DECLINED; store sample when granted
DELETE /inspection/{id}	close session; ephemeral cleanup
POST /chat	LangGraph START → llm_turn → END; evidence-grounded reply

The repository layout is part of the architecture. The tree below is the real project folder structure (abbreviated view at commit 931822c; `__init__.py` package markers, byte-code caches and runtime data directories under `data/`, `datasets/`, `storage/` and `public/` are omitted):

```
apps
├─ api                                # FastAPI service
│  └─ agent                          # LangGraph turns; Groq assistant + StubAssistant
│  └─ inspection                     # consent service, inspection context
│  └─ routers                         # health, inspection (photo-first flow), chat
│  └─ shared                         # API response schemas
│  └─ storage                         # SQLite + image/session/state/training stores
│  └─ vision                         # capture-quality gate
│  └─ container.py                   # dependency container + model resolution/caching
│  └─ model_catalog.py               # five-entry catalogue (ADR 0012)
│  └─ errors.py                      # API error taxonomy
│  └─ main.py                        # app entrypoint + health/metadata
│  └─ settings.py                    # environment settings
└─ web                               # Next.js 16 demo client
   └─ app                            # pages: / (landing), /demo (inspection composer)
   └─ components                     # CinematicIntro, BrandMark, ChatPanel, ConsentBanner, DemoJourney
   └─ e2e                           # Playwright specs (cinematic, composer, journey, responsive)
   └─ lib                            # typed API client; cinematic canvas/preload/sequence

ml
├─ inference                         # preprocess, engine, features, overlay, classes, errors
├─ models                           # resnet34_unet, hybrid_segmentation (legacy arch-tagged files)
├─ training                         # train harness, loss, EMA, class stats, dataset
├─ analysis                         # offline checkpoint/metadata analysis
├─ evaluation                       # metrics, evaluate_run, small_damage
└─ experiments                      # run records + checkpoints (omitted from listing)

docs
├─ architecture                     # overall + CNN-Transformer segmentation design
├─ decisions                         # ADRs 0001-0013 (scope, lineage and catalogue authority)
```

```

└─ ml                                # ML engineering guidelines
└─ research                          # research report, problem definition, alignment, IEEE review
tests                               # 28 pinned test modules (CI quality gates)
pyproject.toml uv.lock requirements-ci.txt conftest.py

```

These folders mirror the actual repository at commit fb2fb59, the working tree that produced the experiments and screenshots in this report. apps/api is split into routers, vision, agent, inspection, shared and storage layers so the capture-quality gate, the photo-first flow and the LLM stay cleanly separated, and model_catalog.py keeps the runtime model list in one auditable place; apps/web holds the Next.js client with its own Playwright end-to-end suites; ml keeps inference, models, training, analysis and evaluation disjoint so the training harness, the inference engine and the metrics cannot silently diverge; docs/decisions holds the thirteen ADRs that fix scope, model lineage and the model catalogue; and tests pins the behaviour this report claims - quality, preprocessing equivalence, features, overlay, engine dispatch, the model catalogue, schemas, storage and the assistant honesty contract.

4.4 DATA FLOW / PROCESSING PIPELINE

The photo-first pipeline is a strict, test-pinned sequence of stages, each with a single accept/reject decision. The user photographs the damage, the photo is uploaded as an image asset and immediately evaluated by the capture-quality gate; only a passing photo reaches the segmentation model. The model output is then reduced to evidence - detected classes, connected-component instances, image-denominator area ratios and per-instance confidence; the evidence is handed to the assistant, which may only describe it; the user is then asked for an explicit consent decision, and closing the session triggers ephemeral cleanup. Figure 1.1 diagrams this contract and Figure 4.1 shows the layered architecture behind it.

A failed quality gate short-circuits the analysis: the API returns `QUALITY_FAILED` with the measured signals (sharpness variance, mean luminance, bright fraction, contrast) and retake guidance, and no segmentation is attempted. This is deliberate - running a segmentation model on a blurred, dark or glary photo would produce confident but meaningless masks, which would break the evidence-honesty contract.

The binding sequence implemented by the router is shown below. UploadResponse, AnalyzeResponse, InspectionStateResponse, ConsentResponse and ChatResponse are all typed in apps/api/shared/schemas.py; Appendix B, Listing B.9 reproduces the analyse flow that returns them.

```

POST /inspection/session      -> create session (uuid)
POST /inspection/{id}/upload  -> store one photo as an image asset
POST /inspection/{id}/analyze -> quality gate -> segmentation -> evidence -> assistant
GET  /inspection/{id}        -> read current state (quality / inspection / messages)
POST /inspection/{id}/consent -> GRANTED persists a labelled training sample
DELETE /inspection/{id}      -> close session; ephemeral cleanup
POST /chat                   -> LangGraph START -> llm_turn -> END (evidence-grounded)

```

4.5 ML / CV METHODOLOGY

The machine-learning methodology follows the locked architecture specification and the project's ground-truth labelling policy. Training uses CarDD (2816 training, 810 validation and 374 test images, six damage classes plus background); the class distribution is severely imbalanced (crack is a minority class), which motivates inverse-frequency class weighting and class-aware sampling. Part-segmentation masks were excluded per ADR 0009, and no true physical area in cm² is ever claimed - damage area is reported strictly as a fraction of the image, which is all one uncontrolled photograph can justify. ADR 0003 requires inference preprocessing to be bit-equivalent to training preprocessing; tests/test_inference_preprocess.py pins that equivalence.

The training objective is a deep-supervised combination of soft Dice loss and Focal loss (0.5 Dice + 0.5 Focal, gamma 2, inverse-frequency class weights with a fixed 0.1 background weight). The main decoder output and two auxiliary supervision heads are weighted 0.75 / 0.15 / 0.10. Training uses AdamW (backbone learning rate 1e-4, head 3e-4, weight decay 1e-4), a 5-epoch linear warm-up then cosine decay to 1e-6, exponential moving-average (EMA 0.999) weight averaging, frozen BatchNorm statistics in the encoder, training-only augmentation (horizontal flip, affine rotation/scale), mixed precision and a batch of four at 512x512. Appendix B reproduces the loss, EMA and metrics code verbatim.

Two architectures share the same encoder/decoder trunk. The baseline is a ResNet-34 U-Net (ResNet34UNet). The proposed model (HybridSegmentation) reuses the same ResNet-34 encoder and decoder but inserts a bottleneck Transformer at the coarsest level: the 512-channel 16x16 feature map is projected 1x1 to 256 channels (256 tokens with a learned 2-D positional embedding), passes through four pre-normalised Transformer blocks (four heads, FFN 1024, GELU, dropout 0.10), and is fused back into the U-Net decoder, which keeps deep supervision on its main and two auxiliary heads. The objective is the deep-supervised soft-Dice and Focal combination fixed for the current catalogue under ADR 0013; the earlier

cross-entropy objective (ADR 0008) is historical and is not reintroduced. The controlled comparison of the two architectures at two seeds each is reported in Chapter 6, together with the limits of that contract.

Evaluation is deliberately honest about absent classes: per-class and mean metrics are computed from the confusion matrix (Appendix B, Listing B.8), and any class that never appears in either prediction or target is scored 0.0 rather than skipped. The model card shipped inside the engine records the artefact's provenance - training run, epochs, parameters, validation metrics - so every deployed checkpoint carries its own honesty metadata.

4.6 USER INTERFACE AND INTERACTION DESIGN

The user interface design follows the photo-first narrative: the home page opens with a scroll-driven cinematic sequence that dramatises the inspection journey, then hands over to the composer, where the user attaches a single photo (optionally with a message). The conversation thread renders each analysis as an assistant message, enriched with damage-class chips, mean confidence, image-area fraction and the mandatory “machine predictions, not verified damage” disclaimer. Failures (quality rejection) and successes produce visibly different UI states, and training-consent is offered in-line and always optional. Chapter 5 reproduces the implemented screens.

4.7 NOVELTY

The project's novelty lies in combining a photo-first scope with evidence-labelled confidence, enforced by decision records rather than by convention. Most published car-damage tools print damage percentages and repair costs as facts; this system labels every number - model prediction, derived feature or measured signal - and refuses cost and repair questions by design (ADR 0011). The capture-quality gate before segmentation is itself a contribution: instead of degrading a bad photo into confident nonsense, the system asks for a better picture.

Scientifically, the work compares a ResNet-34 U-Net baseline with the proposed bottleneck-Transformer hybrid on an identical CarDD split, training schedule, objective and seed pair (42 and 1337), which keeps the head-to-head interpretable while making the run-to-run spread visible instead of hiding it behind a single seed. The hybrid injects global context at the coarsest resolution with a modest parameter and VRAM increase (27.9M versus 24.6M parameters; about 2.0–2.2 versus 1.9 GB peak VRAM). The catalogue is itself a contribution: five explicitly registered model IDs with a recorded status, a product default and a server-side

model-locked inference path (ADR 0012), so no result can be attributed to an unnamed checkpoint.

Operationally, the consent pipeline turns everyday inspections into a consent-gated training-data flywheel: granted samples are stored separately from session data with provenance `MODEL_SUGGESTED` and await human verification before ever entering a future training set. Finally, the engineering practice documented throughout this report - decision records, a pin-equivalence test, complete verbatim code listings and CI quality gates - makes every claim reproducible from the repository itself.

4.8 SUMMARY

The design separates what is measured from what is inferred at every layer, keeps the inference path dependent on training code or data only through locked, tested equivalence, and makes honesty (labels, disclaimers, consent) a first-class functional requirement rather than a documentation afterthought.

CHAPTER 5 TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 OUTLINE

This chapter describes how each design element was implemented. Code excerpts are reproduced from the repository and every listing is labelled with its source file. The final section walks through the demonstration from the supplied screenshots, with the on-screen values reproduced exactly.

5.2 TECHNICAL CODING AND CODE SOLUTIONS

5.2.1 Inference preprocessing (ml/inference/preprocess.py)

Inference must reproduce the training transform bit-for-bit; ADR 0003 pins this equivalence and a test recomputes the training ops independently. The decode and preprocess path is reproduced verbatim in Appendix B, Listing B.1.

Complete verbatim code is reproduced in Appendix B, Listing B.1.

5.2.2 Capture-quality gate (apps/api/vision/quality.py)

The capture-quality heuristic validator runs before any model call and never inspects model output; rejected photos return only guidance. It is reproduced verbatim in Appendix B, Listing B.2.

Complete verbatim code is reproduced in Appendix B, Listing B.2.

5.2.3 Analysis endpoint (apps/api/routers/inspection.py)

The core of the /analyze flow — quality gate first, then engine inference, then feature extraction and evidence persistence — is reproduced verbatim in Appendix B, Listing B.9.

Complete verbatim code is reproduced in Appendix B, Listing B.9.

5.2.4 Front-end photo handling (apps/web/components/demo/DemoJourney.tsx)

On the client, the composer uploads the photo and immediately calls /analyze; the reply is rendered as an assistant message with overlay and honest labels. The typed API client is reproduced verbatim in Appendix B, Listing B.14, and the full component source lives at apps/web/components/demo/DemoJourney.tsx.

Complete verbatim listings: the typed API client is reproduced in Appendix B, Listing B.14; the full component source lives at apps/web/components/demo/DemoJourney.tsx.

5.3 PROTOTYPE SUBMISSION

The working prototype was exercised end-to-end with the six photographs supplied for this report. Figures 5.1–5.3 show the home page and composer; Figures 5.4–5.5 show the quality gate rejecting a blurred photo; Figures 5.6–5.15 show the five analysis scenes. Every screenshot below is a real browser capture of the running system, and the numbers reported are the ones the assistant actually produced.



Fig. 5.1 First frame of the scroll-driven cinematic home-page intro (frame 1/757).

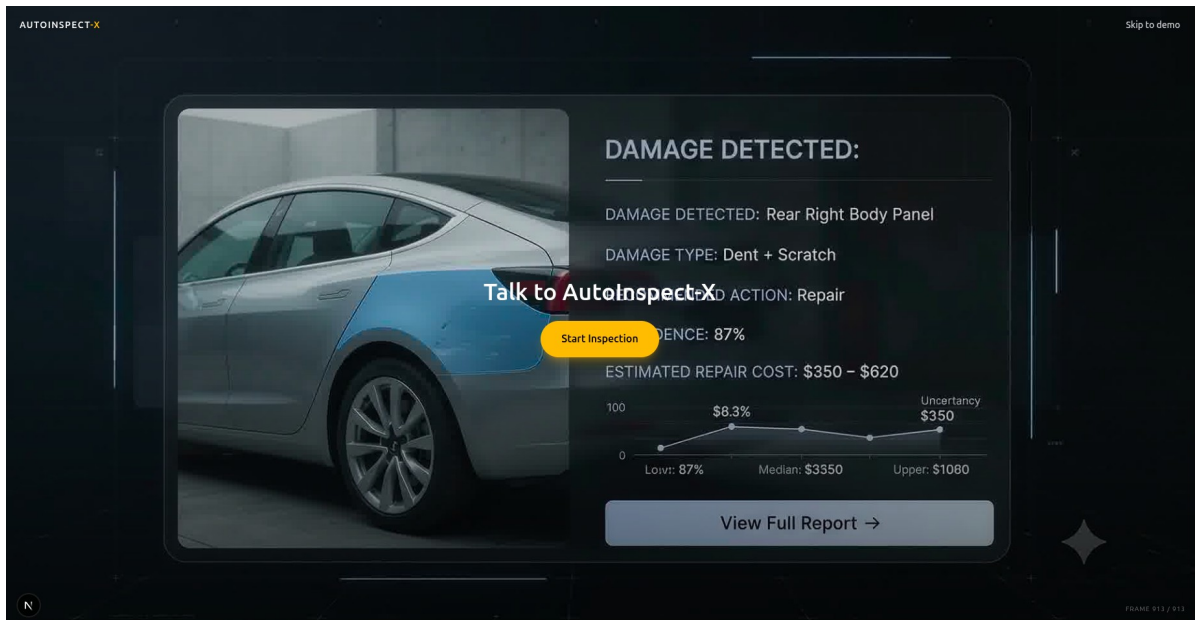


Fig. 5.2 Near-end frame of the cinematic intro sequence (frame 757/757).

The cinematic intro scrolls a pre-rendered 757-frame 30 FPS image sequence drawn to a single high-DPI canvas (frame 1/757 in Fig. 5.1, frame 757/757 in Fig. 5.2), with a prefer-

reduced-motion static fallback. The timeline is four scenes of 300, 240, 133 and 84 frames; every live frame was audited frame by frame, and the 156 unused frames of the final scene were moved to archive/ because they still carried pre-ADR 0011 copy. The shipped sequence therefore contains no repair-cost line, in line with the photo-first scope. Figure 5.3 shows the composer that follows.

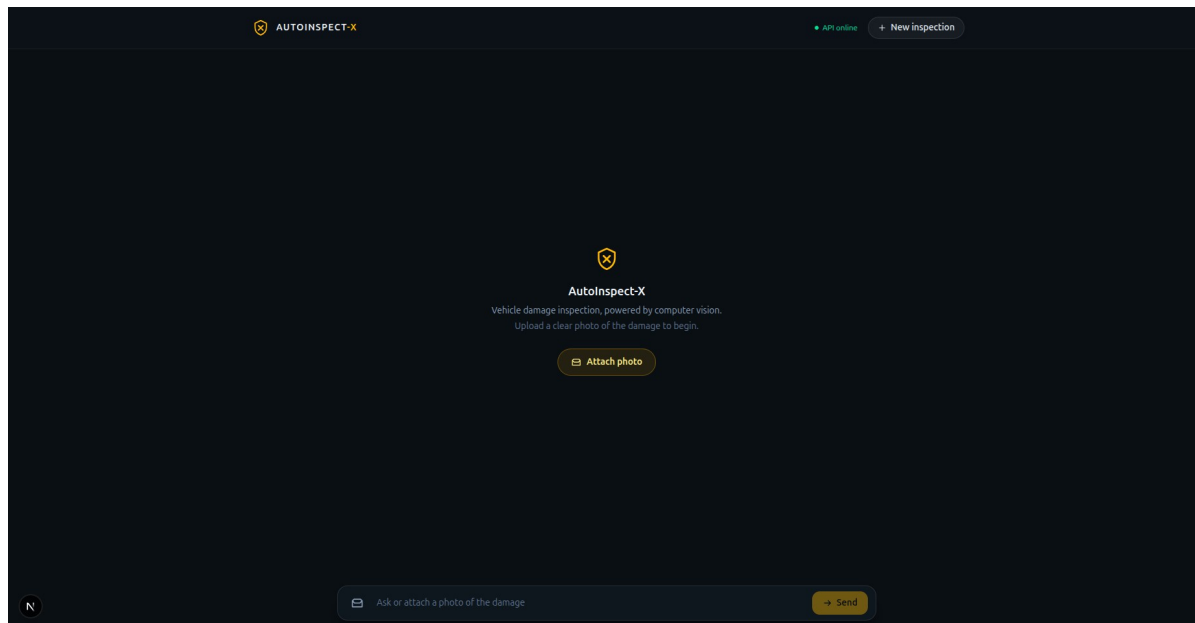


Fig. 5.3 Inspection composer at /demo with upload prompt and free-text field.

The quality gate was exercised with a deliberately blurred photograph (Fig. 5.4). The validator measured sharpness_variance 1.22, mean_luminance 139.28, bright_fraction 0.0 and contrast_std 74.08 – dominated by the sharpness failure against the 25.0 threshold – and the endpoint returned QUALITY_FAILED with retake guidance, producing no mask (Fig. 5.5).



Fig. 5.4 Blurred input photograph used to exercise the capture-quality gate.

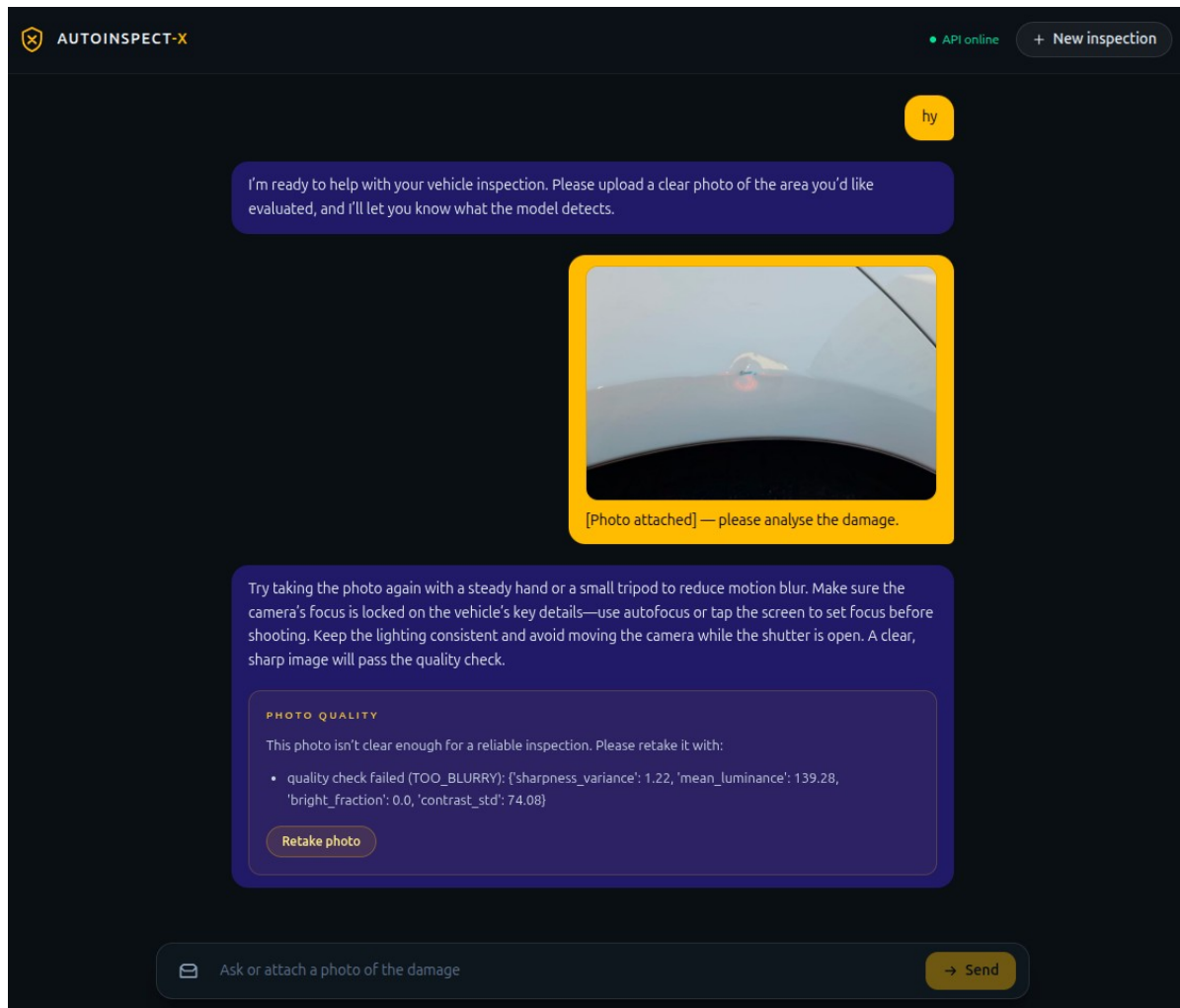


Fig. 5.5 Capture-quality rejection (TOO_BLURRY) with retake guidance; no mask is produced. Five valid photographs then exercised the full pipeline: a minor scratch (Figs. 5.6–5.7), a minor dent/scratch (Figs. 5.8–5.9), a medium multi-damage scene (Figs. 5.10–5.11), a major dent (Figs. 5.12–5.13) and a multi-cue (“confusing”) photo (Figs. 5.14–5.15).



Fig. 5.6 Minor-scratch test photograph (street scene).

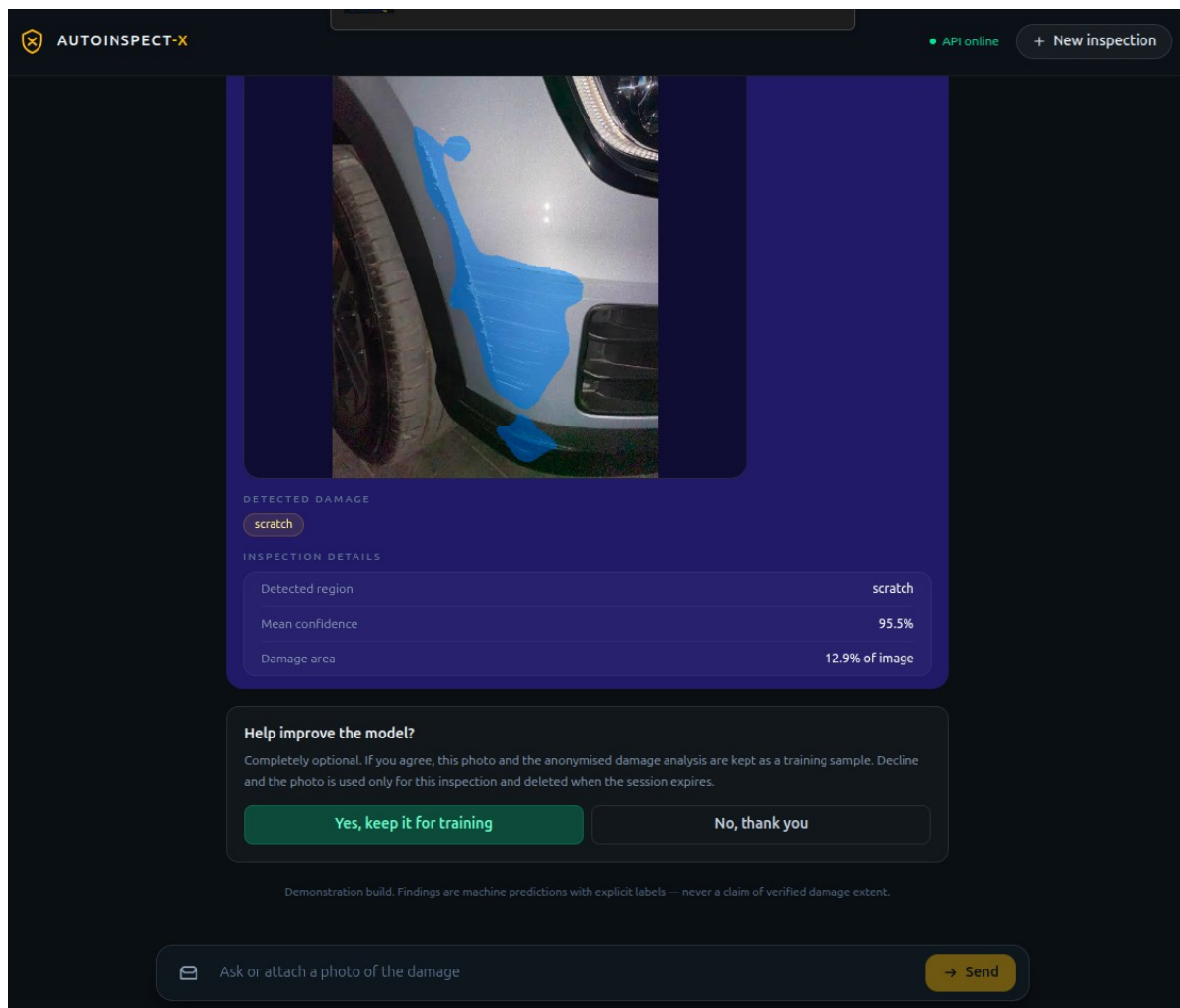


Fig. 5.7 Minor-scratch result - detected region, mean confidence, image-area fraction, consent prompt.



Fig. 5.8 Minor dent/scratch test photograph (close-up).

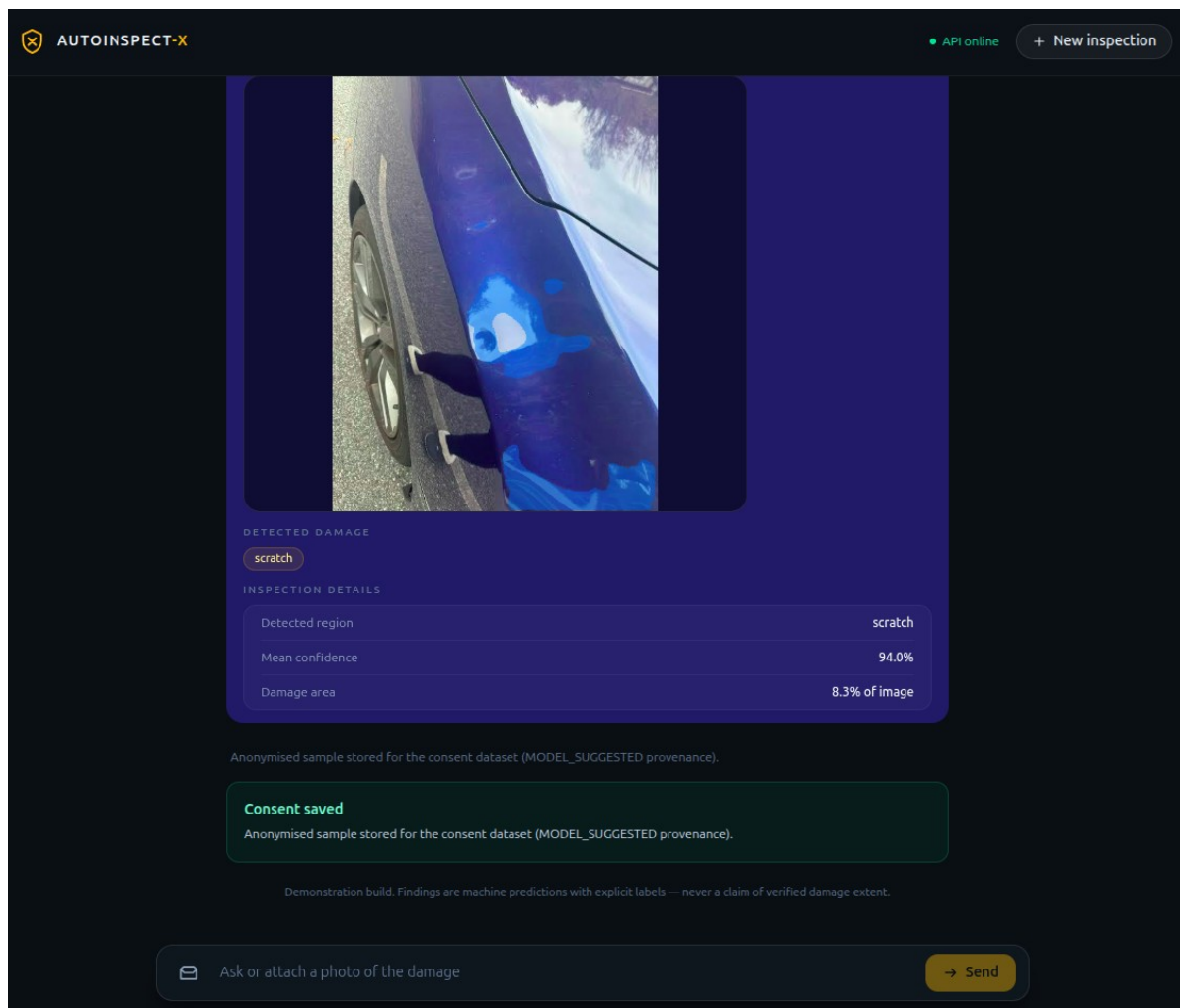


Fig. 5.9 Minor-damage result - scratch chip with confidence, damage-area ratio, consent granted.



Fig. 5.10 Medium-damage test photograph.

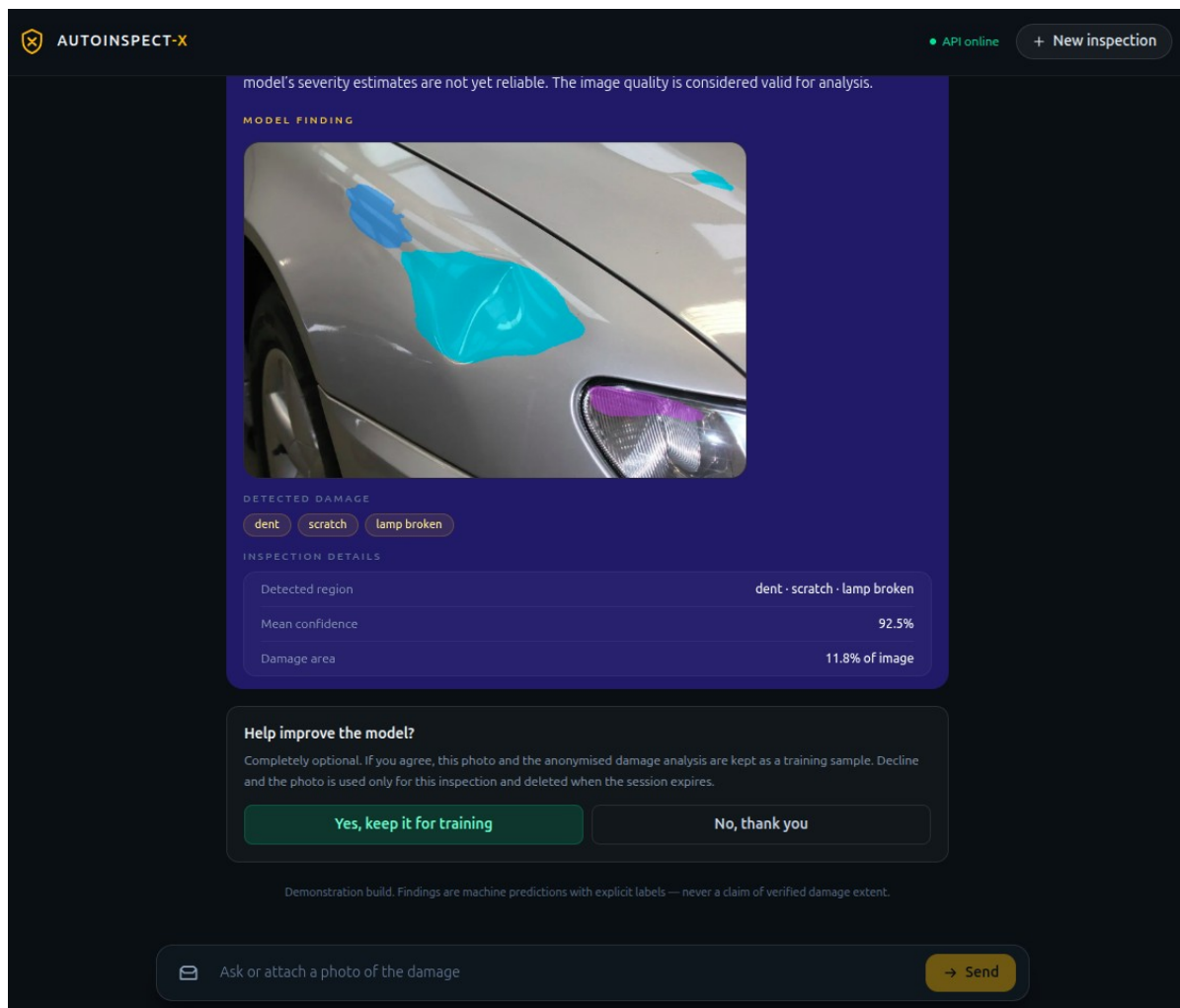


Fig. 5.11 Medium-damage result - dent, scratch and lamp-broken chips with consent prompt.



Fig. 5.12 Major-damage test photograph.

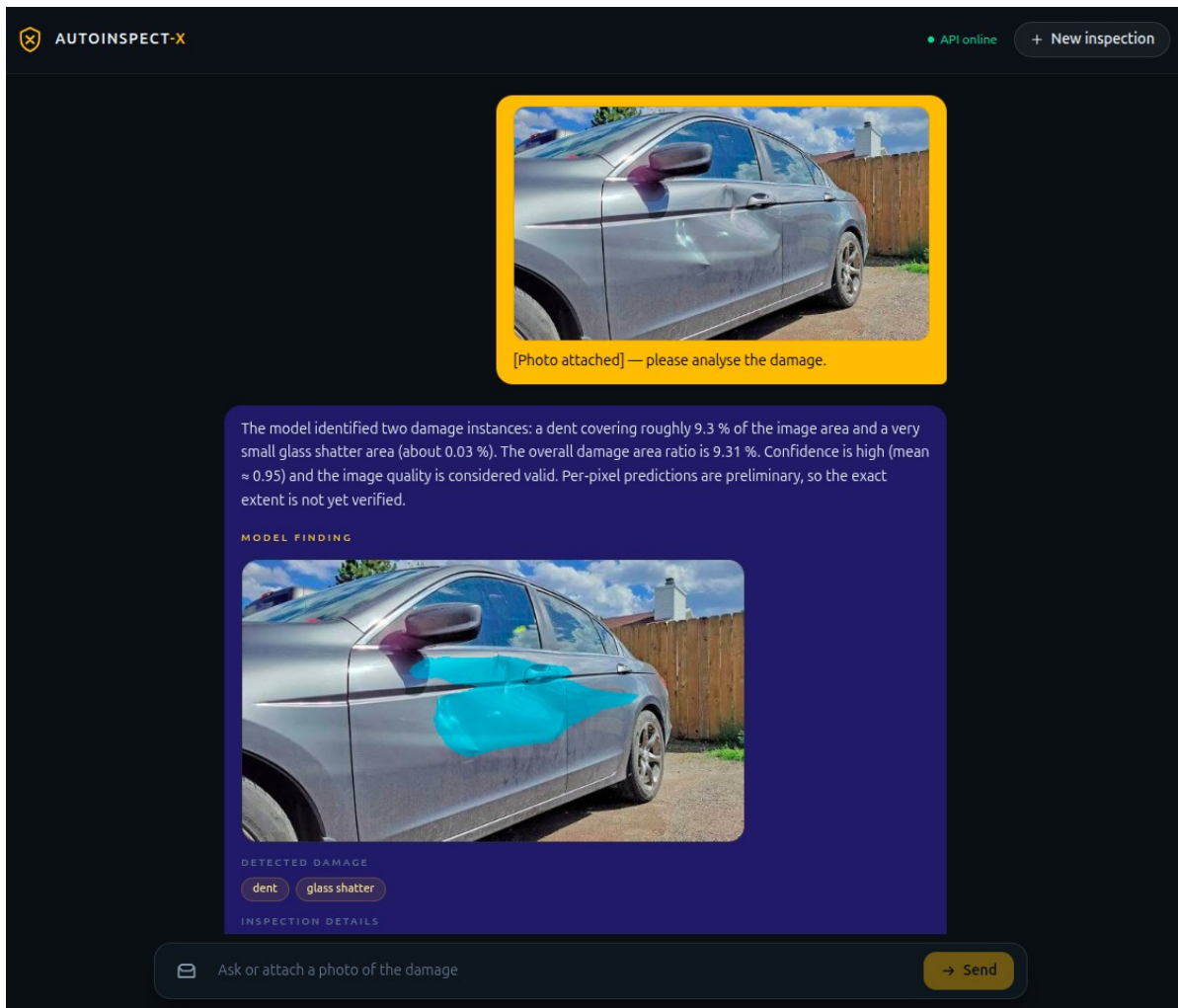


Fig. 5.13 Major-damage result - dent with a small glass-shatter region and consent prompt.



Fig. 5.14 Multi-cue (confusing) test photograph.

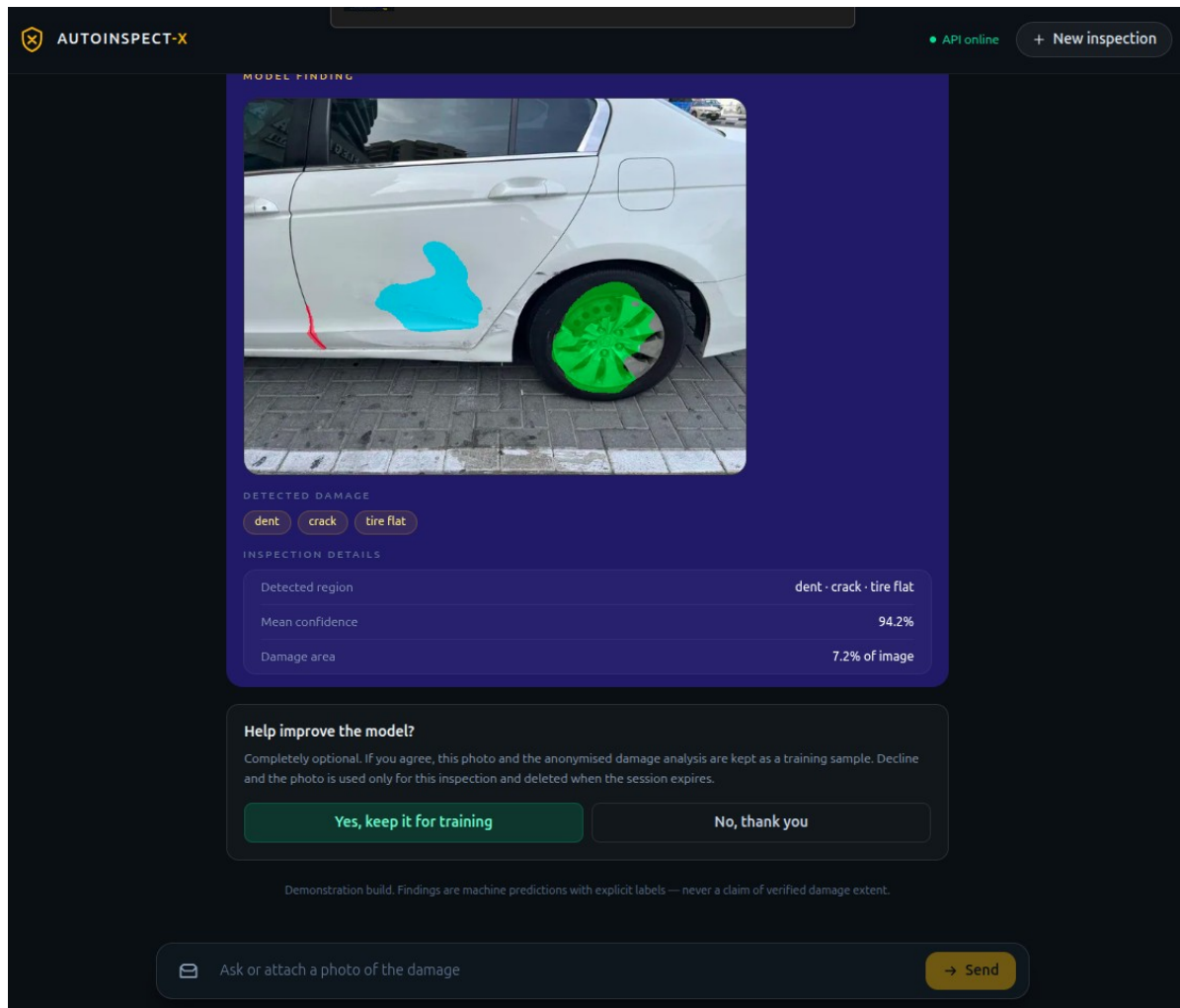


Fig. 5.15 Multi-cue result - dent, crack and tire-flat chips with mean confidence.

The observed outputs are summarised in Table 5.1 (blur rejection) and Table 5.2 (analysis scenes).

Metric	Measured	Threshold
sharpness_variance	1.22	min 25.0 (failed)
mean_luminance	139.28	min 45.0 (ok)
bright_fraction	0.00	max 0.30 (ok)
contrast_std	74.08	min 12.0 (ok)
status	TOO_BLURRY	QUALITY_FAILED → guidance only

Table 5.1 — capture-quality outcome for the blurred photo (on-screen values).

Scene	Classes detected	Damage area (Σ of image)	Mean conf.	Notes
MinorScratch	scratch	12.9%	95.5%	consent offered
minor dent	scratch (4 instances)	8.3%	94.0%	consent granted; sample saved
medium dent	dent, scratch, lamp	11.8% (8.9/1.5/1.4)	92.5%	valid; consent

	broken (4 inst.)			offered
Major dent	dent, glass shatter	9.31% (9.3/0.03)	94.8–95.0%	re-analysis identical
Confusing	dent, crack, tire flat	7.2% (2.9/0.12/4.2)	94.2%	quality VALID

Table 5.2 — demonstration analysis outcomes (values as rendered by the assistant).

Note how the assistant’s narration stays inside the evidence: it repeats area ratios and confidence, and it appends the disclaimer that masks are preliminary and severity estimates are not yet reliable. The consent flow shows “granted – sample stored (MODEL_SUGGESTED provenance)” for the minor-dent scene and an optional decline path elsewhere (additional screenshots in Appendix A).

5.4 SUMMARY

The prototype is complete to the product’s Phase R: a working photo-first demo with honest labels, offline-safe assistant, consent pipeline, end-to-end tests, Playwright suites and CI quality gates. Two deliberate engineering rules are enforced by tests – no cost/repair/quote field anywhere, and no session photo may be used for training without consent.

CHAPTER 6 PROJECT OUTCOME AND APPLICABILITY

6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

The measurable core of the project is a five-entry model catalogue served by the API (ADR 0012). Four entries form the controlled comparison: a ResNet-34 U-Net baseline and the CNN+Transformer hybrid, each trained at seeds 42 and 1337 under one locked 60-epoch configuration, so the architecture is the only systematic difference. The fifth entry, `final100_hybrid_seed42`, is an exploratory hybrid continuation that was reconstructed from surviving checkpoints and is explicitly marked incomplete. The product default is `final60_hybrid_seed42`. Everything around them – quality gate, features, assistant, consent, UI, tests – is already described in Chapter 5.

6.2 SIGNIFICANT PROJECT OUTCOMES

Table 6.2 lists the catalogue as it is registered in the repository, with the state of each record. The two controlled seeds per architecture are a deliberate contract: they expose run-to-run spread without implying a completed three-seed study or a statistical test.

Model ID	Status	Best val fg mIoU	Best epoch	Record state
<code>final60_baseline_seed42</code>	CONTROLLED	0.667659	44	recorded; early stop 54
<code>final60_baseline_seed1337</code>	CONTROLLED	0.668115	50	recorded; 60 epochs, no early stop
<code>final60_hybrid_seed42</code>	CONTROLLED	0.670167	49	recorded; early stop 59; product default
<code>final60_hybrid_seed1337</code>	CONTROLLED	0.673167	47	recorded; early stop 57; resume metadata retained
<code>final100_hybrid_seed42</code>	EXPLORATORY	0.671550	58	partial; completed: false

Table 6.2 — the five-entry model catalogue (ADR 0012). The exploratory entry carries `record_status PARTIAL_RECONSTRUCTED_FROM_CHECKPOINTS` with `completed: false`.

Table 6.3 gives the configuration locked for the four controlled runs; only the model ID, the architecture and the seed differ between them.

Configuration	<code>final60_baseline_seed{42,1337}</code>	<code>final60_hybrid_seed{42,1337}</code>
Model	ResNet34UNet (pretrained encoder)	HybridSegmentation (ResNet34 + bottleneck Transformer)
Parameters	24,577,757	27,933,661
Image size / batch	512×512 / effective 4	512×512 / effective 4
Optimizer / LR	AdamW; backbone 1e-4, head 3e-4	AdamW; backbone 1e-4, head 3e-4
Schedule	5-epoch warmup + cosine → 1e-6	5-epoch warmup + cosine → 1e-6
Loss	0.5 Dice + 0.5 Focal ($\gamma=2$), deep-sup 0.75/0.15/0.10	same
Class weights	inverse-frequency (fg); background 0.1	same

Regularization	EMA 0.999, weight decay 1e-4, mixed precision	same
Data	train-only augmentation: h-flip, affine (rot $\pm 8^\circ$, scale 0.9–1.1), brightness/contrast	same
Peak VRAM	≈ 1.88 GB	≈ 2.01 – 2.24 GB
Batches / epoch	704	704

Table 6.3 — locked training configuration (source: ml/experiments/*/run_record.json).

Table 6.4 reports the best validation metrics of the controlled runs (foreground = damage classes only; EMA checkpoint).

Val metric	baseline seed 42	baseline seed 1337	hybrid seed 42	hybrid seed 1337
Foreground mIoU	0.667659	0.668115	0.670167	0.673167
Foreground mDice	0.784076	0.785013	0.784787	not retained in the record
Pixel accuracy	0.923544	0.924811	0.924496	not retained in the record
Best epoch	44	50	49	47
Epochs run	55	60	60	58 (3 retained records)

Table 6.4 — best validation metrics per controlled run. The seed-1337 hybrid record was reconstructed from surviving checkpoints, so only its best-epoch mIoU and epoch index were retained; the missing columns are marked instead of being estimated.

Readings: (i) minority classes remain the hardest – crack (651 training instances) stays near 0.37–0.39 IoU, glass shatter exceeds 0.94 and dent stays around 0.51–0.53, so small dents and cracks are not reliably separated; (ii) the hybrid leads the baseline on foreground mIoU at both seeds, by about 0.0025 and 0.0051, but the per-seed spread within each architecture is of the same order, so the two-seed contract does not establish a winner and no significance test is claimed; (iii) the exploratory 100-epoch hybrid reached 0.671550 at epoch 58, inside the range of the controlled runs, but its record is incomplete, so it is not used as evidence. The model card shipped inside the engine records each checkpoint’s provenance and its own validation metrics rather than a single headline number.

Class	baseline s42	baseline s1337	hybrid s42
background	0.9102	0.9117	0.9108
dent	0.5128	0.5264	0.5260
scratch	0.5698	0.5787	0.5798
crack	0.3917	0.3937	0.3664
glass shatter	0.9482	0.9452	0.9459
lamp broken	0.7227	0.6953	0.7382
tire flat	0.8606	0.8694	0.8648

Table 6.5 — per-class validation IoU at the best epoch. Only the three runs whose epoch-level metrics were retained can be shown; the hybrid seed-1337 record has no per-class breakdown.

6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

The project outcomes are applicable in several near-term real-world contexts: first-party photo triage for insurance (deciding which claims deserve a desk review before a human adjuster or garage inspection); rental-car return documentation with a photographic, labelled damage record; workshop intake where the honest quality-gate-rejected photo prompts the technician to ask for a better picture rather than guess; and vision-training-data collection in which, with consent, everyday inspections become ground truth. The consent pipeline is the flywheel: the subset of samples explicitly offered by users is stored separately from session data and is ready to be passed through human verification before joining any future training set (provenance MODEL_SUGGESTED).

6.4 INFERENCE

The project outcomes are applicable in several near-term real-world contexts: first-party photo triage for insurance (deciding which claims deserve a desk review before a human adjuster or garage inspection); rental-car return documentation with a photographic, labelled damage record; workshop intake where the honest quality-gate-rejected photo prompts the technician to ask for a better picture rather than guess; and vision-training-data collection in which, with consent, everyday inspections become ground truth. The consent pipeline is the flywheel: the subset of samples explicitly offered by users is stored separately from session data and is ready to be passed through human verification before joining any future training set (provenance MODEL_SUGGESTED).

CHAPTER 7 CONCLUSIONS AND RECOMMENDATION

7.1 OUTLINE

The project delivered a working, honest, photo-first vehicle damage inspection system: a capture-quality gate, a CNN+Transformer hybrid segmentation model compared with a ResNet-34 U-Net baseline on CarDD, evidence-labelled features, a grounded conversational assistant, an opt-in consent pipeline, a polished Next.js demo and CI quality gates. Where the literature and common tools present damage percentages as facts and repair costs as predictions, this system prints “machine prediction, preliminary, not verified damage extent” and refuses cost questions by design – a contract enforced by tests.

7.2 LIMITATION/CONSTRAINTS OF THE SYSTEM

Constraints and limitations are documented honestly throughout: the controlled catalogue has two seeds per architecture, which is enough to expose run-to-run spread but not to establish an architecture winner, and a third seed is still outstanding; the exploratory 100-epoch hybrid record is partial and is excluded from the comparison; minority classes (crack, small dent instances) are not reliably separated; damage area is image-denominator only and physically calibrated cm^2 is unavailable without a calibration method; the demonstration photographs are illustrative scenes, not a validated external test set, and a leakage-free external bench is planned but missing; the LLM depends on Groq availability (the stub is the offline fallback) and can still paraphrase evidence inaccurately even with grounding; Live inference runs on a single 4 GB laptop GPU; the frozen test split has not been used for the reported validation numbers, so no test-set generalisation claim is made; and the per-class analysis is limited to the three runs whose epoch-level metrics were retained.

7.3 FUTURE ENHANCEMENTS

Recommended future work: (1) complete the third seed for both architectures and close the hybrid_seed1337 record so the comparison rests on a full contract rather than a partial one; (2) run the frozen CarDD test split once, after the catalogue is final, and report it as the only generalisation measurement; (3) build the planned external, leakage-free validation set with verified labels; (4) add geometric calibration so a calibrated physical area becomes defensible where conditions allow; (5) human-in-the-loop verification of consented samples to lift MODEL_SUGGESTED provenance; (6) multi-view and temporal hints for occluded damage; (7) deployment hardening (container images, model registry, observability) and a

production consent/privacy review; (8) ablations on transformer blocks/heads and fusion, which the current contract deliberately leaves as future work; and (9) an accessibility and localisation pass on the demo.

7.4 INFERENCE

Inference: the photo-first contract is sound and implementable today. An honest-quality system – gate first, segment with a tractable model, label every number as a derived, not measured, value, ground the assistant, and ask before keeping the data – beats a flashier one that fabricates certainty. The foundation laid in Phase-I is the correct base for the longer training runs, external validation and human-in-the-loop refinement that define Phase-II.

APPENDIX A SCREENSHOTS

This appendix reproduces the supplementary browser captures. Figures 5.1–5.15 in Chapter 5 already show the primary screens; the figures here show the composer pre-analysis state and the assistant narratives and consent outcomes for each scene, without duplicating any figure used earlier.

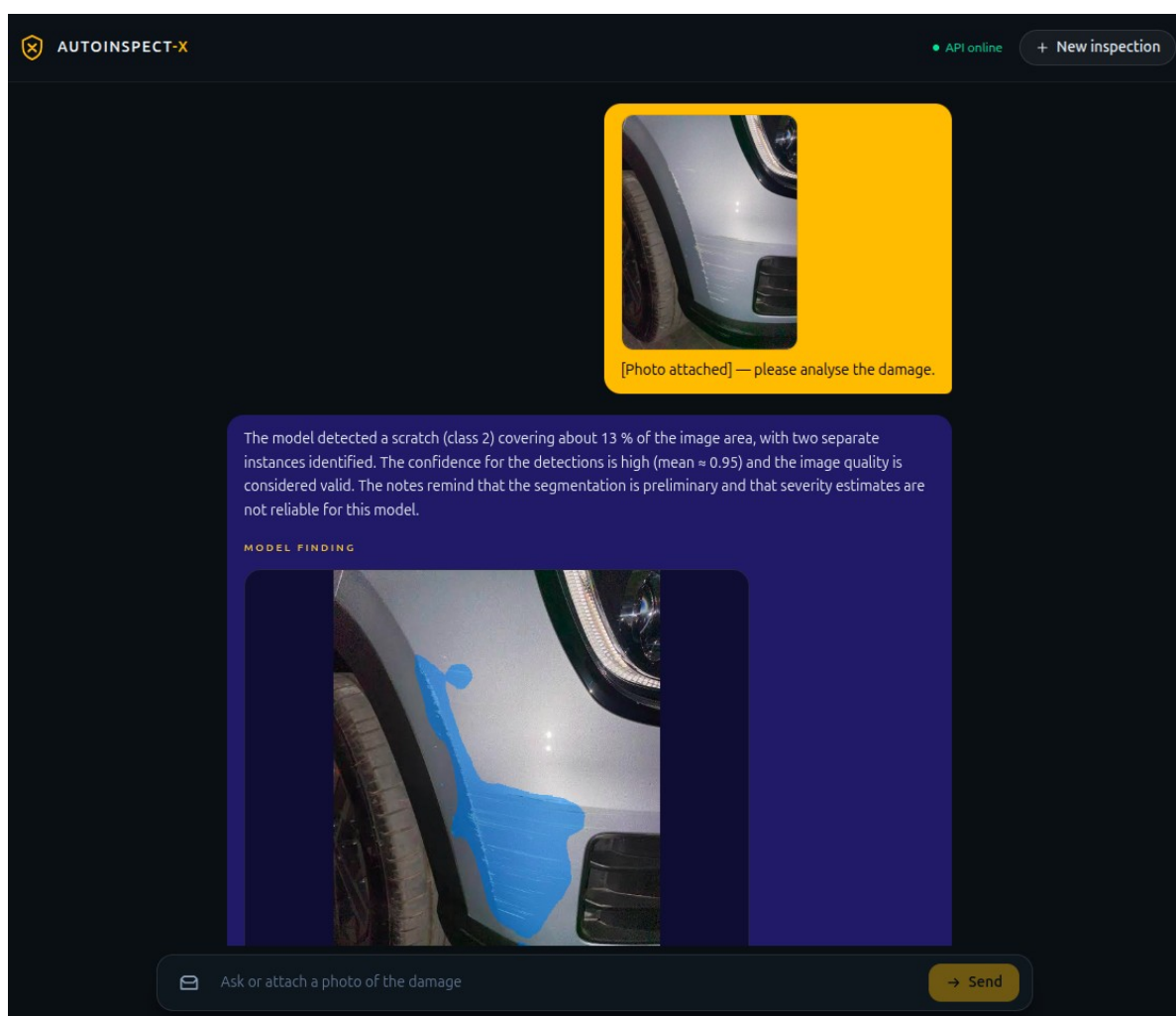


Fig. A.1 Composer with the photo attached before analysis (minor-scratch session).

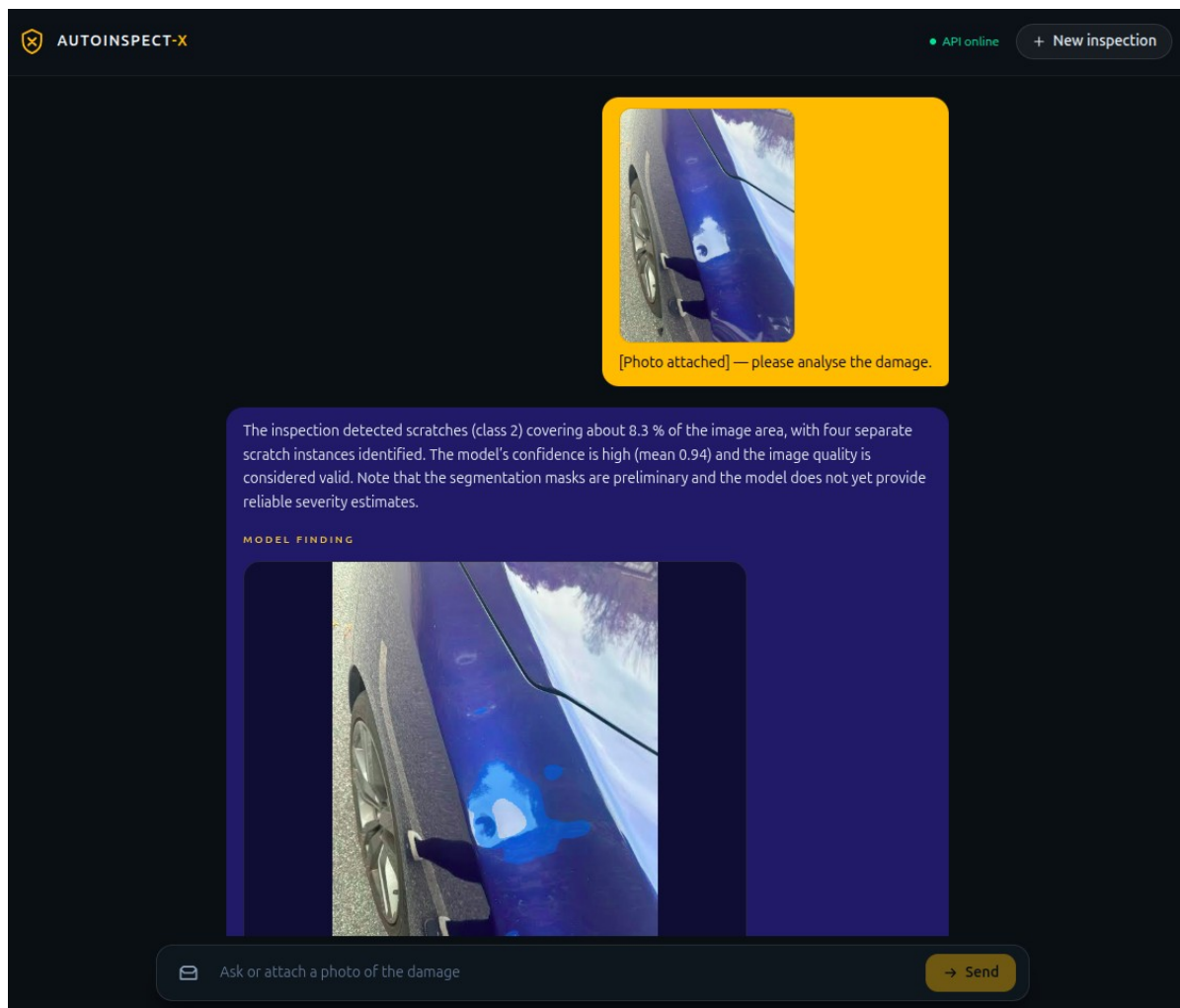


Fig. A.2 Assistant narrative for a minor dent (the assistant's “Here is what the visual model identified” summary).

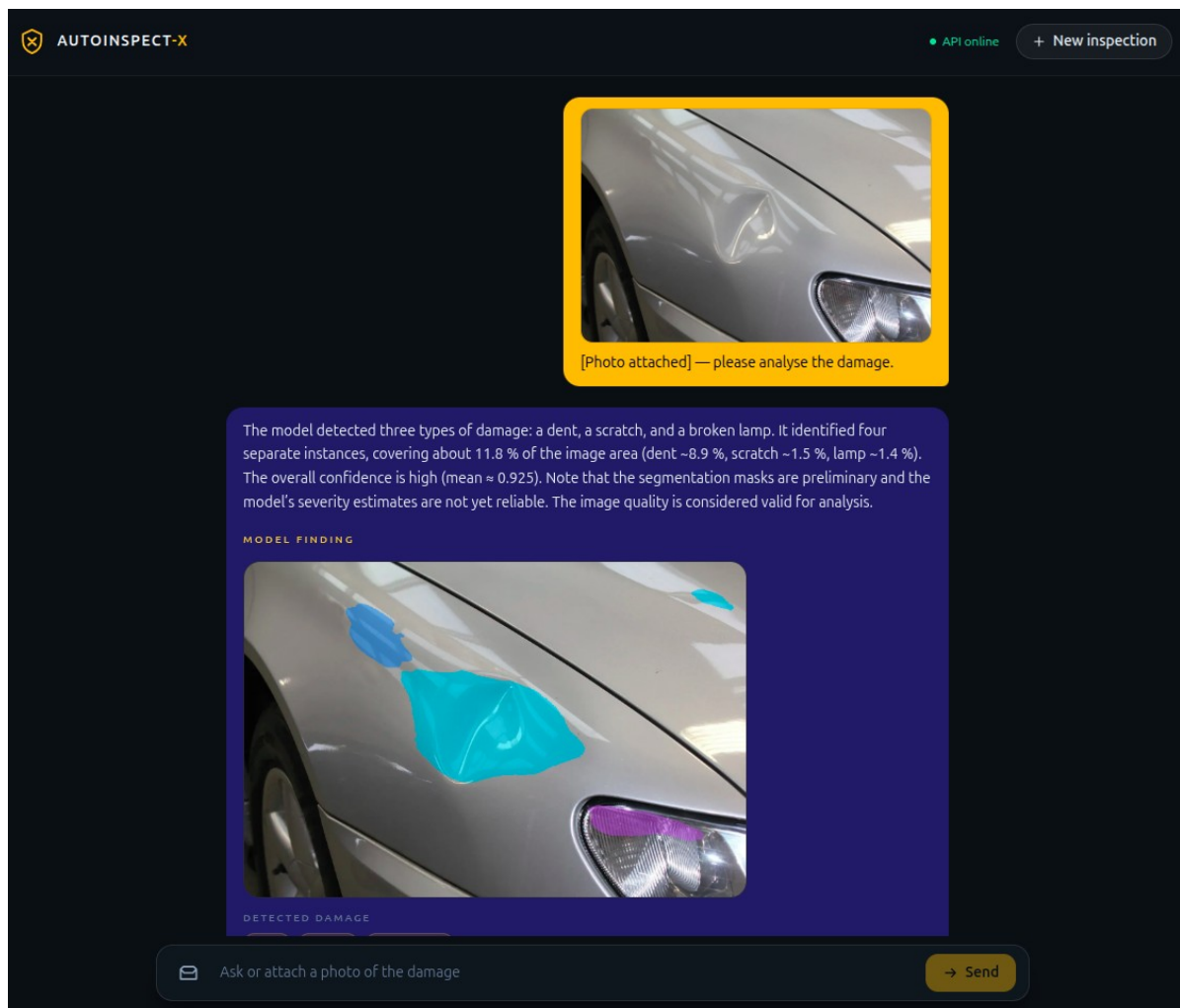


Fig. A.3 Assistant narrative for a medium-damage photograph.

API online
+ New inspection

DETECTED DAMAGE

dent glass shatter

INSPECTION DETAILS

Detected region	dent - glass shatter
Mean confidence	94.8%
Damage area	9.3% of image

re-check

Re-analysis produced the same results: two detected instances—a dent covering about 9.3 % of the image and a very small glass shatter area (~0.03 %). The overall damage area ratio remains 9.31 %. Confidence is high (mean ≈ 0.95) and the image quality is valid. Per-pixel predictions are still preliminary, so the exact extent is not yet verified.

Help improve the model?

Completely optional. If you agree, this photo and the anonymised damage analysis are kept as a training sample. Decline and the photo is used only for this inspection and deleted when the session expires.

Yes, keep it for training
No, thank you

Demonstration build. Findings are machine predictions with explicit labels — never a claim of verified damage extent.

Ask or attach a photo of the damage

→ Send

Fig. A.4 Major-damage result with consent granted and re-analysis confirmation.

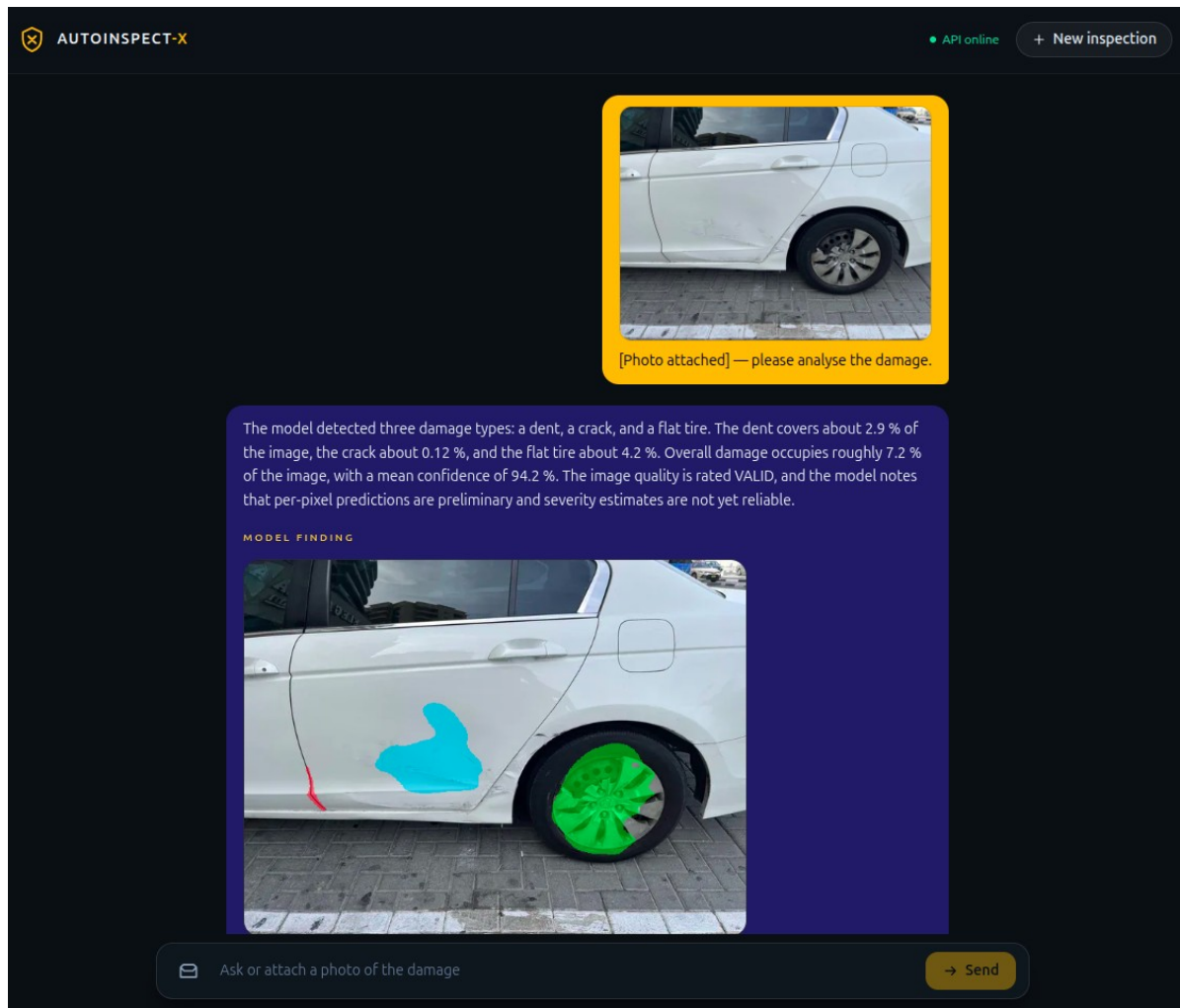


Fig. A.5 Assistant narrative for the multi-cue (confusing) photograph.

APPENDIX B CODE LISTINGS

Every listing below is a complete, verbatim excerpt extracted directly from the Git working tree at the commit this report was verified against; nothing is synthetic or paraphrased. Each caption gives the exact source path and the line range in that file, so every excerpt can be reproduced on disk. Excerpts were chosen to be self-contained (a function, class or constant) rather than mid-expression cuts.

Listing B.1 - Photograph loading and inference preprocessing | File: ml/inference/preprocess.py | Lines: 25-49

```
def load_image_rgb(source: Path | bytes) -> np.ndarray:
    """Decode a JPEG/PNG/WebP image as RGB uint8 ``HxWx3``."""
    if isinstance(source, Path):
        raw = np.fromfile(str(source), dtype=np.uint8)
        if raw.size == 0:
            raise ValueError(f"empty or unreadable image file: {source}")
    elif isinstance(source, bytes):
        raw = np.frombuffer(source, dtype=np.uint8)
    else:
        raise TypeError(f"unsupported image source: {type(source).__name__}")
    decoded = cv2.imdecode(raw, cv2.IMREAD_COLOR)
    if decoded is None:
        raise ValueError("could not decode image bytes")
    return cv2.cvtColor(decoded, cv2.COLOR_BGR2RGB)

def preprocess_image(rgb: np.ndarray) -> torch.Tensor:
    """Convert RGB uint8 HWC into the training-time float32 CHW tensor."""
    if rgb.dtype != np.uint8:
        raise ValueError(f"preprocess expects uint8 RGB, got dtype {rgb.dtype!r}")
    if rgb.ndim != 3 or rgb.shape[2] != 3:
        raise ValueError(f"preprocess expects HxWx3 RGB, got shape {rgb.shape}")
    resized = cv2.resize(rgb, (TARGET_SIZE, TARGET_SIZE), interpolation=cv2.INTER_LINEAR)
    normalized = resized.astype(np.float32) / 255.0
    return torch.from_numpy(np.transpose(normalized, (2, 0, 1))).contiguous()
```

Listing B.2 - Capture-quality validator (quality gate before segmentation) | File:
apps/api/vision/quality.py | Lines: 57-93

```
class ImageQualityValidator:
    """Heuristic validator; never inspects model output."""

    def __init__(self, thresholds: QualityThresholds | None = None) -> None:
        self._thresholds = thresholds or QualityThresholds()

    def assess(self, rgb: np.ndarray) -> QualityResult:
        if rgb.ndim != 3 or rgb.shape[2] != 3:
            raise ValueError(f"expected RGB HxWx3, got shape {rgb.shape}")
        gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
        sharpness = float(cv2.Laplacian(gray, cv2.CV_64F).var())
        metrics = {
            "sharpness_variance": round(sharpness, 2),
            "mean_luminance": round(float(gray.mean()), 2),
            "bright_fraction": round(float((gray > 250).mean()), 4),
            "contrast_std": round(float(gray.std()), 2),
        }

        status = QualityStatus.VALID
        if metrics["mean_luminance"] < self._thresholds.min_mean_luminance:
            status = QualityStatus.TOO_DARK
        elif metrics["bright_fraction"] > self._thresholds.max_bright_fraction:
            status = QualityStatus.EXCESSIVE_GLARE
        elif sharpness < self._thresholds.min_sharpness_variance:
            status = QualityStatus.TOO_BLURRY
        elif metrics["contrast_std"] < self._thresholds.min_contrast_std:
            status = QualityStatus.INSUFFICIENT_CONTEXT

        reasons: list[str] = []
        if status is not QualityStatus.VALID:
            reasons.append(f"quality check failed ({status.value}): {metrics}")
        return QualityResult(
```

```

        status=status,
        is_valid=status is QualityStatus.VALID,
        metrics=metrics,
        reasons=reasons,
    )

```

Listing B.3 - Segmentation-engine architecture dispatch | File: ml/inference/engine.py | Lines: 72-92

```

def _build_model(arch: str, *, base: int, num_classes: int) -> torch.nn.Module:
    """Instantiate the checkpoint's architecture from its ``model_arch`` key.

    Legacy baselines predate the key (or store ``cardd_unet``) and map to
    CarddUNet; hybrid runs store ``cardd_hybrid``; the current research
    baseline/hybrid store ``resnet34_unet`` / ``hybrid`` (architecture spec v3).
    The pretrained encoder is deliberately not downloaded here – the checkpoint
    ``model_state`` is authoritative and replaced immediately after. Unknown
    keys fail loudly so an artefact is never silently run with the wrong
    architecture.
    """
    arch_l = "".join(arch.lower().split("_"))
    if arch_l in {"resnet34unet", "baseline"}:
        return ResNet34UNet(num_classes=num_classes, pretrained=False)
    if arch_l in {"hybrid", "hybridsegmentation"}:
        return HybridSegmentation(num_classes=num_classes, pretrained=False)
    if arch_l in {"carddhybrid"}:
        return CarddHybrid(base=base, num_classes=num_classes)
    if not arch or arch_l in {"carddunet"}:
        return CarddUNet(base=base, num_classes=num_classes)
    raise ModelVersionError(f"unknown model_arch in checkpoint: {arch!r}")

```

Listing B.4 - Damage instance feature extraction (connected components; image-denominator area) | File: ml/inference/features.py | Lines: 24-137

```

class DamageInstance:
    """One connected damage region discovered in the predicted mask."""

    instance_id: int
    class_id: int

```

```

class_name: str
area_pixels: int
area_ratio_image: float
bbox: tuple[int, int, int, int]
centroid: tuple[float, float]
mean_confidence: float
max_confidence: float

def to_dict(self) -> dict[str, Any]:
    return {
        "instance_id": self.instance_id,
        "class_id": self.class_id,
        "class_name": self.class_name,
        "area_pixels": self.area_pixels,
        "area_ratio_image": self.area_ratio_image,
        "bbox": {"x": self.bbox[0], "y": self.bbox[1], "w": self.bbox[2], "h": self.bbox[3]},
        "centroid": {"x": round(self.centroid[0], 2), "y": round(self.centroid[1], 2)},
        "mean_confidence": self.mean_confidence,
        "max_confidence": self.max_confidence,
    }

@dataclass(frozen=True)
class DamageFeatures:
    """The honest, serialisable feature set for one analysed image."""

    width: int
    height: int
    instances: list[DamageInstance]
    damage_area_ratio_image: float
    per_class_area_ratio_image: dict[int, float]
    classes_present: dict[int, str]
    num_instances: int
    low_confidence_instances: int
    mask: np.ndarray

    def to_dict(self) -> dict[str, Any]:
        return {
            "width": self.width,

```

```

        "height": self.height,

        "num_instances": self.num_instances,

        "damage_area_ratio_image": round(self.damage_area_ratio_image, 6),

        "per_class_area_ratio_image": {

            str(cid): round(v, 6) for cid, v in self.per_class_area_ratio_image.items()

        },

        "classes_present": {str(cid): name for cid, name in self.classes_present.items()},

        "instances": [inst.to_dict() for inst in self.instances],

        "low_confidence_instances": self.low_confidence_instances,

    }

def extract_features(

    result: SegmentationResult,

    *,

    min_component_pixels: int = 8,

) -> DamageFeatures:

    """Turn a segmentation result into connected damage regions and features."""

    mask = result.mask

    pixel_confidence = result.pixel_confidence

    h, w = mask.shape

    image_pixels = float(h * w)

    instances: list[DamageInstance] = []

    per_class_area: dict[int, int] = {}

    for class_id in range(1, int(result.prob.shape[0])):

        class_mask = (mask == class_id).astype(np.uint8)

        _, labels, stats, _centroids = cv2.connectedComponentsWithStats(class_mask, connectivity=8)

        labels = cast(np.ndarray, labels)

        stats = cast(np.ndarray, stats)

        for label in range(1, int(stats.shape[0])):

            x, y, box_w, box_h, area = (int(v) for v in stats[label][:5])

            if area < min_component_pixels:

                continue

            comp = labels == label

            weights = pixel_confidence[comp]

            ys, xs = np.nonzero(comp)

            centroid = (float(xs.mean()), float(ys.mean()))

```

```

per_class_area[class_id] = per_class_area.get(class_id, 0) + area
instances.append(
    DamageInstance(
        instance_id=len(instances),
        class_id=class_id,
        class_name=ID_TO_CLASS[class_id],
        area_pixels=area,
        area_ratio_image=round(area / image_pixels, 6),
        bbox=(x, y, box_w, box_h),
        centroid=centroid,
        mean_confidence=round(float(weights.mean()), 4),
        max_confidence=round(float(weights.max()), 4),
    )
)

damage_pixels = sum(per_class_area.values())
per_class_ratio = {cid: area / image_pixels for cid, area in per_class_area.items()}
classes_present = {cid: ID_TO_CLASS[cid] for cid in per_class_area}
low_confidence = sum(1 for inst in instances if inst.mean_confidence < _LOW_CONFIDENCE)

return DamageFeatures(
    width=w,
    height=h,
    instances=instances,
    damage_area_ratio_image=damage_pixels / image_pixels,
    per_class_area_ratio_image=per_class_ratio,
    classes_present=classes_present,
    num_instances=len(instances),
    low_confidence_instances=low_confidence,
    mask=mask.copy(),
)

```

Listing B.5 - HybridSegmentation model: ResNet-34 encoder + bottleneck Transformer | File: ml/models/hybrid_segmentation.py | Lines: 43-113

```

BOTTLENECK_TOKENS = 256 # 16 x 16 token grid
TRANSFORMER_BLOCKS = 4
TRANSFORMER_HEADS = 4
TRANSFORMER_FFN_DIM = 1024
TRANSFORMER_DROPOUT = 0.10

```

```

class BottleneckTransformer(nn.Module):
    """Attention applied only to the 16x16 bottleneck features.

    Input : C5 tensor [B, 512, 16, 16]
    Project: 1x1 conv 512 -> 256, flatten to 256 tokens -> [B, 256, 256]
    Output : reshaped [B, 256, 16, 16] global-context representation
    """

    def __init__(
        self,
        in_channels: int = 512,
        d_model: int = BOTTLENECK_DIM,
        num_blocks: int = TRANSFORMER_BLOCKS,
        num_heads: int = TRANSFORMER_HEADS,
        ffn_dim: int = TRANSFORMER_FFN_DIM,
        dropout: float = TRANSFORMER_DROPOUT,
    ) -> None:
        super().__init__()
        self.d_model = d_model
        self.project = nn.Conv2d(in_channels, d_model, kernel_size=1)
        self.pos_embed = nn.Parameter(torch.zeros(1, BOTTLENECK_TOKENS, d_model))
        nn.init.normal_(self.pos_embed, 0.0, 0.02)
        layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=num_heads,
            dim_feedforward=ffn_dim,
            dropout=dropout,
            activation="gelu",
            norm_first=True,
            batch_first=True,
        )
        self.blocks = nn.TransformerEncoder(layer, num_layers=num_blocks)

    def forward(self, c5: Tensor) -> Tensor:
        x = self.project(c5)

```

```

        batch, channels, height, width = x.shape
        tokens = x.flatten(2).transpose(1, 2)
        tokens = tokens + self.pos_embed
        tokens = self.blocks(tokens)
        out = tokens.transpose(1, 2).reshape(batch, channels, height, width)
        return cast(Tensor, out)

class HybridSegmentation(nn.Module):
    """ResNet34 + bottleneck Transformer + additive fusion + U-Net decoder."""

    def __init__(
        self,
        num_classes: int = NUM_CLASSES,
        pretrained: bool = True,
        weights_path: str | None = None,
    ) -> None:
        super().__init__()
        self.num_classes = num_classes
        self.encoder = ResNet34Encoder(pretrained=pretrained, weights_path=weights_path)
        self.project = nn.Conv2d(512, BOTTLENECK_DIM, kernel_size=1)
        self.transformer = BottleneckTransformer()
        self.decoder = UnetDecoder(num_classes=num_classes)

    def forward(self, x: Tensor) -> tuple[Tensor, Tensor, Tensor]:
        skips = self.encoder(x)
        c5 = skips["c5"]
        fused = self.project(c5) + self.transformer(c5)
        return cast(tuple[Tensor, Tensor, Tensor], self.decoder(fused, skips))

```

Listing B.6 - Deep-supervised Dice + Focal training objective | File: ml/training/loss.py | Lines: 30-202

```

DICE_SMOOTH = 1e-6 # soft Dice smoothing (architecture spec 19A)
FOCAL_GAMMA = 2.0 # Focal loss gamma (architecture spec 19A)
DEEP_SUPERVISION_WEIGHTS = (0.75, 0.15, 0.10) # main, aux1, aux2
BACKGROUND_PIXEL_WEIGHT = 0.1 # fixed reduced background weight (spec 19A)

```

```

@dataclass(frozen=True)
class ClassWeights:
    """Pixel-class weights for the Focal term, derived from the train split.

    ``foreground`` maps CarDD category id (1..6) to its weight; ``background``
    is the fixed reduced weight applied to class 0.

    """

    foreground: dict[int, float]
    background: float = BACKGROUND_PIXEL_WEIGHT

    def as_tensor(self, num_classes: int, device: torch.device) -> Tensor:
        weights = torch.full((num_classes,), 1.0, device=device)
        weights[0] = self.background
        for class_id, value in self.foreground.items():
            if 1 <= class_id < num_classes:
                weights[class_id] = value
        return weights

def aggregate_targets(masks: Tensor, labels: Tensor, num_classes: int) -> Tensor:
    """Aggregate per-instance masks into per-class binary targets (B, C, H, W).

    `masks` is (B, N, H, W) uint8 per-instance; `labels` is (B, N) int64 with
    -1 marking padded instances. Padded instances have all-zero masks, so OR
    aggregation ignores them. Class channels are indexed by the dataset
    category id; channel 0 (background) is never set (its pixels are the
    complement of all damage channels).

    """
    b, _n, h, w = masks.shape
    targets = torch.zeros(b, num_classes, h, w, device=masks.device, dtype=masks.dtype)
    for i in range(_n):
        cls = labels[:, i]
        m = masks[:, i]
        for c in range(1, num_classes):
            sel = cls == c
            if bool(sel.any()):
                targets[sel, c] = torch.maximum(targets[sel, c], m[sel])
    return targets

```

```

def class_targets(masks: Tensor, labels: Tensor, num_classes: int) -> Tensor:
    """Return the argmax class-index target (B, H, W) long, background = class 0.

    Built from `aggregate_targets`, so overlapping damage classes collapse to a
    single label per pixel – the same decode the evaluation harness uses.
    """
    return aggregate_targets(masks, labels, num_classes).argmax(dim=1)


def cross_entropy_loss(
    logits: Tensor,
    masks: Tensor,
    labels: Tensor,
    num_classes: int,
) -> Tensor:
    """Softmax cross-entropy between logits and the argmax class target.

    Background is class 0. Uses the same decode as the evaluation harness
    (argmax), so training and metric semantics match. Uniform weighting is the
    default baseline; a weighted variant can be added behind an ADR later.
    """
    targets = class_targets(masks, labels, num_classes)
    return F.cross_entropy(logits, targets)


def resize_targets_nearest(targets: Tensor, size: tuple[int, int]) -> Tensor:
    """Nearest-neighbour resize of per-class target channels to `size`.

    Used to bring full-resolution targets to the auxiliary-head resolutions
    (architecture spec 8 / 19A). Returns the input unchanged when already at
    the right resolution.
    """
    if targets.shape[-2:] == size:
        return targets
    return cast(Tensor, F.interpolate(targets, size=size, mode="nearest"))

```

```

def _soft_dice_loss(logits: Tensor, targets: Tensor, smooth: float = DICE_SMOOTH) -> Tensor:
    """Soft multi-class Dice loss over the six foreground classes only.

    Background channel 0 is excluded from the mean (architecture spec 19A).
    """
    probs = torch.softmax(logits, dim=1)
    num_classes = int(logits.shape[1])
    losses: list[Tensor] = []
    for class_id in range(1, num_classes):
        pred = probs[:, class_id]
        target = targets[:, class_id]
        numerator = 2.0 * (pred * target).sum() + smooth
        denominator = pred.sum() + target.sum() + smooth
        losses.append(1.0 - numerator / denominator)
    if not losses:
        return torch.zeros((), device=logits.device, dtype=logits.dtype)
    return torch.stack(losses).mean()

def _focal_loss(
    logits: Tensor,
    targets_idx: Tensor,
    weights: Tensor,
    gamma: float = FOCAL_GAMMA,
) -> Tensor:
    """Multi-class Focal loss (architecture spec 19A), averaged over pixels."""
    num_classes = int(logits.shape[1])
    probs = torch.softmax(logits, dim=1)
    log_probs = torch.log_softmax(logits, dim=1)
    cls_idx = targets_idx.clamp(min=0, max=num_classes - 1)
    one_hot = F.one_hot(cls_idx, num_classes).permute(0, 3, 1, 2).to(dtype=probs.dtype)
    pt = (probs * one_hot).sum(dim=1)
    log_pt = (log_probs * one_hot).sum(dim=1)
    class_weight = weights[cls_idx]
    focal = -(class_weight * (1.0 - pt).pow(gamma) * log_pt)
    return cast(Tensor, focal.mean())

def level_loss(

```

```

logits: Tensor,

targets: Tensor,

weights: Tensor,

smooth: float = DICE_SMOOTH,

gamma: float = FOCAL_GAMMA,

) -> Tensor:

    """0.5 Dice + 0.5 Focal for one prediction scale (architecture spec 19A)."""

    targets_idx = targets.argmax(dim=1)

    dice = _soft_dice_loss(logits, targets, smooth=smooth)

    focal = _focal_loss(logits, targets_idx, weights, gamma=gamma)

    return 0.5 * dice + 0.5 * focal


def dice_focal_loss(

    main_logits: Tensor,

    aux1_logits: Tensor | None,

    aux2_logits: Tensor | None,

    masks: Tensor,

    labels: Tensor,

    num_classes: int,

    weights: ClassWeights,

    smooth: float = DICE_SMOOTH,

    gamma: float = FOCAL_GAMMA,

) -> tuple[Tensor, dict[str, float]]:

    """Deep-supervised Dice + Focal training objective.

    Sums the per-level objective with the locked weights 0.75 / 0.15 / 0.10.
    Auxiliary targets are nearest-neighbour resized to the head resolution.
    `aux1_logits` / `aux2_logits` may be ``None`` (legacy single-output
    models, in which case only the main objective is used).

    Returns the total loss plus a per-level breakdown for logging.

    """

    targets = aggregate_targets(masks, labels, num_classes).float().to(main_logits.device)

    weight_tensor = weights.as_tensor(num_classes, main_logits.device)

    main_scale = level_loss(main_logits, targets, weight_tensor, smooth=smooth, gamma=gamma)

    total = DEEP_SUPERVISION_WEIGHTS[0] * main_scale

    parts: dict[str, float] = {"main": float(main_scale.item())}

```

```

if aux1_logits is not None:
    t1 = resize_targets_nearest(targets, (int(aux1_logits.shape[2]), int(aux1_logits.shape[3])))
    a1 = level_loss(aux1_logits, t1, weight_tensor, smooth=smooth, gamma=gamma)
    total = total + DEEP_SUPERVISION_WEIGHTS[1] * a1
    parts["aux1"] = float(a1.item())

if aux2_logits is not None:
    t2 = resize_targets_nearest(targets, (int(aux2_logits.shape[2]), int(aux2_logits.shape[3])))
    a2 = level_loss(aux2_logits, t2, weight_tensor, smooth=smooth, gamma=gamma)
    total = total + DEEP_SUPERVISION_WEIGHTS[2] * a2
    parts["aux2"] = float(a2.item())

return total, parts

```

Listing B.7 - Exponential moving-average (EMA) weight averaging | File: ml/training/ema.py | Lines: 19-62

```

class EMA:
    """Maintains decayed copies of all trainable model parameters.

    ``apply_shadow`` / ``restore`` let the trainer evaluate the EMA weights on
    the validation set without losing the in-progress raw optimizer state.
    """

    def __init__(self, model: nn.Module, decay: float = 0.999) -> None:
        self.decay = decay
        self.shadow: dict[str, Tensor] = {}
        self.backup: dict[str, Tensor] = {}
        for name, param in model.named_parameters():
            if param.requires_grad:
                self.shadow[name] = param.detach().clone()

    @torch.no_grad()
    def update(self, model: nn.Module) -> None:
        for name, param in model.named_parameters():
            shadow = self.shadow.get(name)
            if shadow is not None:
                shadow.mul_(self.decay).add_(param.detach(), alpha=1.0 - self.decay)

    @torch.no_grad()
    def apply_shadow(self, model: nn.Module) -> None:

```

```

self.backup.clear()

for name, param in model.named_parameters():
    shadow = self.shadow.get(name)
    if shadow is not None:
        self.backup[name] = param.detach().clone()
        param.data.copy_(shadow)

@torch.no_grad()
def restore(self, model: nn.Module) -> None:
    for name, param in model.named_parameters():
        original = self.backup.pop(name, None)
        if original is not None:
            param.data.copy_(original)

def state_dict(self) -> dict[str, Any]:
    return {"decay": self.decay, "shadow": {k: v.cpu() for k, v in self.shadow.items()}}

def load_state_dict(self, state: dict[str, Any]) -> None:
    self.decay = float(state["decay"])
    self.shadow = {k: v.clone() for k, v in state["shadow"].items()}

```

Listing B.8 - Per-class IoU / Dice / Precision / Recall from the confusion matrix | File: ml/evaluation/metrics.py | Lines: 18-102

```

def confusion_matrix(pred_classes: Tensor, target_classes: Tensor, num_classes: int) -> Tensor:
    """Return a (C, C) confusion matrix over pixel class indices.

    `conf[i, j]` counts pixels whose target class is `i` and predicted class
    is `j`. Inputs are long tensors of shape (B, H, W) indexing classes.
    """
    flat_pred = pred_classes.reshape(-1).long()
    flat_target = target_classes.reshape(-1).long()
    index = flat_target * num_classes + flat_pred
    counts = torch.bincount(index, minlength=num_classes * num_classes)
    return counts.reshape(num_classes, num_classes).float()

def per_class_iou(confusion: Tensor) -> Tensor:
    """Return (C,) IoU per class; absent classes become 0.0."""

```

```

diag = torch.diagonal(confusion)

union = confusion.sum(dim=0) + confusion.sum(dim=1) - diag

present = union > 0

safe_union = torch.where(present, union, torch.ones_like(union))

return torch.where(present, diag / safe_union, torch.zeros_like(diag))

def per_class_dice(confusion: Tensor) -> Tensor:
    """Return (C,) Dice per class; absent classes become 0.0."""
    diag = torch.diagonal(confusion)

    denom = confusion.sum(dim=0) + confusion.sum(dim=1)

    present = denom > 0

    safe_denom = torch.where(present, denom, torch.ones_like(denom))

    return torch.where(present, 2 * diag / safe_denom, torch.zeros_like(diag))

def per_class_precision(confusion: Tensor) -> Tensor:
    """Return (C,) precision per class; classes with no prediction become 0.0.

    precision_c = TP_c / (TP_c + FP_c) = diag_c / column_sum_c.
    """
    diag = torch.diagonal(confusion)

    denom = confusion.sum(dim=0)

    present = denom > 0

    safe_denom = torch.where(present, denom, torch.ones_like(denom))

    return torch.where(present, diag / safe_denom, torch.zeros_like(diag))

def per_class_recall(confusion: Tensor) -> Tensor:
    """Return (C,) recall per class; classes with no target become 0.0.

    recall_c = TP_c / (TP_c + FN_c) = diag_c / row_sum_c.
    """
    diag = torch.diagonal(confusion)

    denom = confusion.sum(dim=1)

    present = denom > 0

    safe_denom = torch.where(present, denom, torch.ones_like(denom))

    return torch.where(present, diag / safe_denom, torch.zeros_like(diag))

```

```

def mean_iou(confusion: Tensor, exclude_background: bool = True) -> float:
    """Macro mean IoU over classes, excluding background by default."""
    scores = per_class_iou(confusion)
    if exclude_background:
        scores = scores[1:]
    return float(scores.mean().item()) if scores.numel() else 0.0

def mean_dice(confusion: Tensor, exclude_background: bool = True) -> float:
    """Macro mean Dice over classes, excluding background by default."""
    scores = per_class_dice(confusion)
    if exclude_background:
        scores = scores[1:]
    return float(scores.mean().item()) if scores.numel() else 0.0

def mean_precision(confusion: Tensor, exclude_background: bool = True) -> float:
    """Macro mean precision over classes, excluding background by default."""
    scores = per_class_precision(confusion)
    if exclude_background:
        scores = scores[1:]
    return float(scores.mean().item()) if scores.numel() else 0.0

def mean_recall(confusion: Tensor, exclude_background: bool = True) -> float:
    """Macro mean recall over classes, excluding background by default."""
    scores = per_class_recall(confusion)
    if exclude_background:
        scores = scores[1:]
    return float(scores.mean().item()) if scores.numel() else 0.0

```

Listing B.9 - Photo-first analyse flow (quality gate -> segmentation -> evidence -> assistant) | File: apps/api/routers/inspection.py | Lines: 192-302

```

def analyze_photo(session_id: str, request: Request) -> AnalyzeResponse:
    c = _valid_session(request, session_id)
    state = _load_state(c, session_id)
    model_id = state.get("model_id") or c.model_id_for_session()
    asset_id = state.get("image_asset_id")
    asset = c.images.get(asset_id) if asset_id else None

```

```

if asset is None:
    _raise(problem(404, APIErrorCode.NO_UPLOADED_PHOTO, "no uploaded photo for this session"))

data = c.images.read(asset)

rgb = load_image_rgb(data)
quality = ImageQualityValidator().assess(rgb)
quality_dict = quality.to_dict()

user_photo_msg = {
    "role": "user",
    "content": "[Photo uploaded] – please analyse the damage.",
}
messages = [*state.get("messages", []), user_photo_msg]

# Capture-quality gate (ADR 0007 contract): reject before any model inference
# on dark/glare/blurry/low-contrast shots. These are user-fixable capture
# problems, not model output. Guidance comes from the assistant; no mask is
# ever produced for a rejected photo.
if not quality.is_valid:
    guidance, fallback = _llm_retake_guidance(c, quality.reasons)
    state.update(
        {
            "messages": [*messages, {"role": "assistant", "content": guidance}],
            "quality": quality_dict,
            "inspection": None,
            "explanation": guidance,
        }
    )
    _save_state(c, session_id, state)
    return AnalyzeResponse(
        session_id=session_id,
        status="QUALITY_FAILED",
        model_id=model_id,
        assistant_message=guidance,
        assistant_fallback=fallback,
        asset_id=asset.asset_id,
        quality_status=quality.status.value,
        quality_reasons=quality.reasons,
    )

```

```

try:
    engine = c.engine(model_id)
except ModelLoadError as exc:
    logger.exception("segmentation engine unavailable (MODEL_UNAVAILABLE): %s", exc)
    _raise(problem(500, APIErrorCode.MODEL_UNAVAILABLE, "model unavailable"))
except ModelVersionError as exc:
    logger.exception("segmentation model version mismatch (MODEL_UNAVAILABLE): %s", exc)
    _raise(problem(500, APIErrorCode.MODEL_UNAVAILABLE, "model unavailable"))
except Exception:
    logger.exception("segmentation engine could not be built")
    _raise(problem(500, APIErrorCode.MODEL_UNAVAILABLE, "model unavailable"))

try:
    result = engine.predict_bytes(data)
except Exception:
    logger.exception("hybrid model forward pass failed")
    _raise(problem(500, APIErrorCode.INFERENCE_FAILED, "model inference failed"))

features = extract_features(result)

overlay_png = encode_png(render_overlay(rgb, result.mask))
overlay_b64 = base64.b64encode(overlay_png).decode("ascii")
c.images.write(session_id, uuid.uuid4().hex, ImageKind.OVERLAY, overlay_png)

inspection = features.to_dict()
inspection.update(
    {
        "mean_confidence": round(result.mean_confidence, 4),
        "damage_fraction": round(result.damage_fraction, 6),
        "low_confidence": result.quality.low_confidence,
        "model_notes": result.quality.notes,
        "quality": quality_dict,
        "model_metadata": result.metadata.to_dict(),
        "overlay_png_base64": overlay_b64,
    }
)

evidence = persist_evidence({"inspection": inspection})

```

```

explanation, fallback = _llm_explanation(c, evidence or {})

state.update(
    {
        "messages": [*messages, {"role": "assistant", "content": explanation}],
        "quality": quality_dict,
        "inspection": inspection,
        "explanation": explanation,
    }
)

_save_state(c, session_id, state)

return AnalyzeResponse(
    session_id=session_id,
    status="OK",
    model_id=model_id,
    assistant_message=explanation,
    assistant_fallback=fallback,
    asset_id=asset.asset_id,
    quality_status=quality.status.value,
    quality_reasons=quality.reasons,
    inspection=inspection,
    classes_present={str(k): v for k, v in features.classes_present.items()},
    low_confidence=result.quality.low_confidence,
    damage_fraction=result.damage_fraction,
    mean_confidence=result.mean_confidence,
    overlay_png_base64=overlay_b64,
)

```

Listing B.10 - Consent service with provenance MODEL_SUGGESTED | File:
apps/api/inspection/consent_service.py | Lines: 33-100

```

class ConsentService:
    """Records a decision and, on GRANTED, persists one training sample."""

    def __init__(
        self,
        consents: SQLiteConsentStore,
        samples: SQLiteTrainingSampleStore,
        images: FsSQLiteImageStore,
        training_root: Path,
        dataset_version: str,
    ):

```

```

) -> None:

    self._consents = consents

    self._samples = samples

    self._images = images

    self._training_root = training_root

    self._dataset_version = dataset_version


def record_decision(
    self,
    session_id: str,
    decision: ConsentDecision,
    dataset_version: str | None = None,
) -> ConsentRecord:
    return self._consents.record(
        session_id, decision, dataset_version=dataset_version or self._dataset_version
    )


def store_training_sample(
    self,
    context: InspectionContext,
    features: DamageFeatures,
    *,
    consent: ConsentRecord,
    image_asset_id: str,
    provenance: str = "MODEL_SUGGESTED",
) -> dict[str, object]:
    """Persist the image + labels for a granted consent; None result means skipped."""
    if consent.decision is not ConsentDecision.GRANTED:
        return {"saved": False, "reason": "consent not granted"}

    asset = self._images.get(image_asset_id)

    if asset is None:
        return {"saved": False, "reason": "image asset no longer available"}

    image_bytes = self._images.read(asset)

    sample = self._samples.add(
        session_id=context.session_id,
        consent=consent,
        image_asset_id=image_asset_id,
        dataset_version=self._dataset_version,

```

```

        labels=self._build_labels(features),
        provenance=provenance,
    )

    version_dir = self._training_root / self._dataset_version
    version_dir.mkdir(parents=True, exist_ok=True)
    ext = asset.path.suffix or ".png"
    target = version_dir / f"{sample.sample_id}{ext}"
    target.write_bytes(image_bytes)
    return {"saved": True, "sample_id": sample.sample_id, "path": str(target)}

    @staticmethod
    def _build_labels(features: DamageFeatures) -> dict[str, object]:
        classes = {name: _CLASS_NAMES_TO_IDS[name] for name in features.classes_present.values()}
        return {
            "class_ids": classes,
            "num_instances": features.num_instances,
            "damage_area_ratio_image": features.damage_area_ratio_image,
        }

```

Listing B.11 - LangGraph turn workflow (START -> llm_turn -> END) | File: apps/api/agent/graph.py | Lines: 69-88

```

def llm_turn(state: InspectionState, services: Services) -> dict[str, Any]:
    reply = services.assistant.chat_reply(
        history=history_from_state(state),
        evidence=persist_evidence(dict(state)),
    )
    return {"reply": reply}

# ----- #
def _compile(services: Services) -> Any:
    b = StateGraph(InspectionState)
    b.add_node("llm_turn", lambda s: llm_turn(s, services))
    b.add_edge(START, "llm_turn")
    b.add_edge("llm_turn", END)
    return b.compile()

```

```
def build_workflow(services: Services) -> Any:
    """Return a compiled, deterministic LangGraph workflow."""
    return _compile(services)
```

Listing B.12 - SQLite schema for sessions, image assets, consent and training samples | File: apps/api/storage/database.py | Lines: 17-64

```
_SCHEMA = """
CREATE TABLE IF NOT EXISTS sessions (
    id            TEXT PRIMARY KEY,
    status        TEXT NOT NULL,
    created_at    TEXT NOT NULL,
    expires_at    TEXT
);

CREATE TABLE IF NOT EXISTS image_assets (
    id            TEXT PRIMARY KEY,
    session_id    TEXT NOT NULL,
    kind          TEXT NOT NULL,
    relative_path TEXT NOT NULL,
    created_at    TEXT NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_image_assets_session ON image_assets(session_id);

-- Consented training side lives in separate tables so cleanup can never reach it.
CREATE TABLE IF NOT EXISTS consent_records (
    id            TEXT PRIMARY KEY,
    session_id    TEXT NOT NULL,
    decision      TEXT NOT NULL,
    dataset_version TEXT,
    created_at    TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS training_samples (
    id            TEXT PRIMARY KEY,
    session_id    TEXT NOT NULL,
```

```

        consent_id          TEXT NOT NULL,
        dataset_version     TEXT NOT NULL,
        annotation_status   TEXT NOT NULL,
        provenance          TEXT NOT NULL,
        image_asset_id      TEXT NOT NULL,
        labels_json         TEXT NOT NULL,
        created_at          TEXT NOT NULL
    );

CREATE INDEX IF NOT EXISTS idx_training_samples_created
    ON training_samples(created_at DESC);

-- Ephemeral workflow state per session (added Phase K; wiped by cleanup).
CREATE TABLE IF NOT EXISTS session_states (
    id          TEXT PRIMARY KEY,
    state_json TEXT NOT NULL,
    updated_at TEXT NOT NULL
);

```

Listing B.13 - Assistant honesty contract (system prompt + offline StubAssistant) | File: apps/api/agent/assistant.py | Lines: 23-158

```

_PRIMARY_SYSTEM_PROMPT = (
    "You are a professional automotive damage inspection assistant for "
    "AutoInspect-X. You help a user understand what the computer-vision model "
    "found in the photo they uploaded, and you can answer follow-up questions "
    "about the inspection.\n\n"
    "RULES:\n"
    "- Describe ONLY what the structured inspection evidence says.\n"
    "- NEVER invent damage, severity, repair actions, costs, or part prices.\n"
    "- Cost and repair pricing are out of scope; if asked, decline briefly.\n"
    "- Evidence flagged low_confidence is preliminary – say so, never present "
    "it as verified damage.\n"
    "- If no evidence is available yet, remind the user to upload a clear "
    "photo of the damage.\n"
    "- Be concise (2-5 sentences), friendly, and factual. No emojis.\n"
)

_QUALITY_SYSTEM_PROMPT = (

```

```

        "You are a friendly automotive inspection assistant. A user uploaded a "
        "photo that failed the capture-quality gate. Give clear, practical retake "
        "guidance based on the listed reasons. Keep it to 2-4 sentences, direct "
        "and encouraging. Never mention costs, quotes, or repairs."
    )

class AssistantUnavailableError(RuntimeError):
    """Raised when a production LLM call fails; routers surface it as 503."""

class AssistantService(Protocol):
    """The seam a router/graph talks to; implemented by LangChain or a stub."""

    provider: str

    def damages_explanation(self, evidence: dict[str, Any]) -> str:
        """Plain-language summary of the segmentation evidence."""
        ...

    def retake_guidance(self, reasons: list[str]) -> str:
        """Guidance for reshooting a photo that failed the quality gate."""
        ...

    def chat_reply(self, history: list[dict[str, str]], evidence: dict[str, Any] | None) -> str:
        """Reply to a follow-up turn, grounded in the stored evidence."""
        ...

    def evidence_payload(evidence: dict[str, Any]) -> str:
        """Serialise the prompt-safe evidence dict for the model or the stub."""
        return json.dumps(evidence, indent=1, default=str)

# ----- #
# Offline double
# ----- #

_QUALITY_REASONS_GUIDANCE = {

```

```

"TOO_BLURRY": "hold the camera steady and make sure the damage is in focus",
"TOO_DARK": "move to better lighting so the surface is clearly visible",
"EXCESSIVE_GLARE": "shoot at an angle that avoids direct light or reflections",
"WRONG_ANGLE": "shoot more directly at the damaged surface",
"DAMAGE_NOT_VISIBLE": "frame the damaged part more tightly",
"INSUFFICIENT_CONTEXT": "step back a little so the damaged area and its "
"surroundings are both visible",
}

```

```

class StubAssistant:
    """Deterministic offline double used without a Groq key (tests/CI/dev).

    Clearly an offline stand-in: it echoes the evidence and never fabricates
    anything the evidence does not contain. It is not a substitute for the LLM.
    """

    provider = "stub"

    def damages_explanation(self, evidence: dict[str, Any]) -> str:
        classes = evidence.get("classes_present") or {}
        ratios = evidence.get("per_class_area_ratio_image") or {}

        if not classes:
            lines = ["The model did not clearly detect any damage classes in this photo."]
        else:
            lines = []
            for name in set(classes.values()):
                cid = next((k for k, v in classes.items() if v == name), None)
                frac = float(ratios.get(str(cid)) or 0.0)
                lines.append(f"• {name} – about {frac * 100:.1f}% of the image frame")
            lines.insert(0, "Here is what the visual model identified:")
        text = "\n".join(lines)
        text += (
            "\n\nThese predictions are preliminary and not verified damage extent."
            if evidence.get("low_confidence")
            else "\n\nAsk me anything about the damage, or upload a new photo."
        )
        return text

```

```

def retake_guidance(self, reasons: list[str]) -> str:
    raw_tips = [_QUALITY_REASONS_GUIDANCE.get(r) for r in reasons]
    tips = [t for t in raw_tips if t is not None]
    if not tips:
        tips = ["take the photo in bright, even light with the camera steady"]
    why = ", ".join(reasons) if reasons else "the photo quality was too low"
    return (
        f"I couldn't analyse that photo because it was flagged: {why}. "
        f"Please retake it – {'; '.join(tips)} – then upload it again."
    )

def chat_reply(
    self,
    history: list[dict[str, str]], # noqa: ARG002 - protocol signature; unused by the stub
    evidence: dict[str, Any] | None,
) -> str:
    if evidence and (evidence.get("classes_present") or {}):
        names = ", ".join(sorted(set((evidence["classes_present"] or {}).values())))
        low = (
            " The model's confidence is low, so treat these as preliminary."
            if evidence.get("low_confidence")
            else ""
        )
        return (
            f"To recap the model's findings: {names}.{low} Ask me anything "
            "about the damage, or upload a fresh photo for a new look."
        )
    if evidence:
        return (
            "The model found no clearly identifiable damage classes in the "
            "last photo. You can upload another angle for a second look."
        )
    return (
        "I don't have an analysed photo for this session yet. Please attach "
        "a clear photo of the damage so I can take a look."
    )

```

```
# Production (LangChain ChatGroq)
```

```
# ----- #
```

Listing B.14 - Typed web client for the inspection API | File: apps/web/lib/api.ts | Lines: 17-128

```
const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";
```

```
export class ApiError extends Error {
```

```
  constructor(
```

```
    message: string,
```

```
    readonly status: number,
```

```
    readonly body?: unknown,
```

```
  ) {
```

```
    super(message);
```

```
    this.name = "ApiError";
```

```
  }
```

```
}
```

```
async function request<T>(path: string, init?: RequestInit): Promise<T> {
```

```
  let response: Response;
```

```
  try {
```

```
    response = await fetch(`${API_BASE}${path}`, {
```

```
      ...init,
```

```
      headers: {
```

```
        ...(init?.body instanceof FormData ? {} : { "Content-Type": "application/json" }),
```

```
        ...init?.headers,
```

```
      },
```

```
    });
```

```
  } catch {
```

```
    throw new ApiError(
```

```
      `Cannot reach the inspection API at ${API_BASE}. Is the backend running?`,
```

```
      0,
```

```
    );
```

```
  }
```

```
if (!response.ok) {
```

```
  let body: unknown;
```

```
  let detail = `Request failed (${response.status})`;
```

```
  try {
```

```
    body = await response.json();
```

```

    const detailField = (body as { detail?: unknown }).detail;
    if (typeof detailField === "string") detail = detailField;
    else if (typeof detailField === "object" && detailField !== null) {
        const message = (detailField as { message?: unknown }).message;
        detail = typeof message === "string" ? message : JSON.stringify(detailField);
    } else {
        detail = JSON.stringify(detailField);
    }
} catch {
    /* keep the fallback message */
}
throw new ApiError(detail, response.status, body);
}
return (await response.json()) as T;
}

export function getHealth(): Promise<HealthResponse> {
    return request<HealthResponse>("/health");
}

export function getModels(): Promise<ModelsResponse> {
    return request<ModelsResponse>("/models");
}

export function createSession(modelId?: string): Promise<SessionCreated> {
    return request<SessionCreated>("/inspection/session", {
        method: "POST",
        body: modelId ? JSON.stringify({ model_id: modelId }) : undefined,
    });
}

export function selectModel(
    sessionId: string,
    modelId: string,
): Promise<ModelSelectionResponse> {
    return request<ModelSelectionResponse>(`/inspection/${sessionId}/model`, {
        method: "PATCH",
        body: JSON.stringify({ model_id: modelId }),
    });
}

```

```

}

export function getInspection(sessionId: string): Promise<InspectionStateResponse> {
  return request<InspectionStateResponse>(`/inspection/${sessionId}`);
}

export function deleteInspection(sessionId: string): Promise<{ session_id: string; status: string }> {
  return request<{ session_id: string; status: string }>(`/inspection/${sessionId}`, {
    method: "DELETE",
  });
}

export async function uploadPhoto(sessionId: string, file: File): Promise<UploadResponse> {
  const form = new FormData();
  form.append("file", file);
  return request<UploadResponse>(`/inspection/${sessionId}/upload`, { method: "POST", body: form });
}

export function analyzePhoto(sessionId: string): Promise<AnalyzeResponse> {
  return request<AnalyzeResponse>(`/inspection/${sessionId}/analyze`, { method: "POST" });
}

export async function sendChat(sessionId: string, message: string): Promise<ChatResponse> {
  return request<ChatResponse>(`/chat`, {
    method: "POST",
    body: JSON.stringify({ session_id: sessionId, message }),
  });
}

export function sendConsent(
  sessionId: string,
  decision: "GRANTED" | "DECLINED",
): Promise<ConsentResponse> {
  return request<ConsentResponse>(`/inspection/${sessionId}/consent`, {
    method: "POST",
    body: JSON.stringify({ decision }),
  });
}

```

Listing B.15 - Five-entry runtime model catalogue (ids, status, default and record provenance) | File: apps/api/model_catalog.py | Lines: 80-177

```
def _spec(
    model_id: str,
    label: str,
    family: ModelFamily,
    architecture: str,
    seed: int,
    configured_epochs: int,
    status: ModelStatus,
    experiment_id: str,
    description: str,
) -> ModelSpec:
    experiment_dir = REPOSITORY_ROOT / "ml" / "experiments" / model_id
    return ModelSpec(
        model_id=model_id,
        label=label,
        family=family,
        architecture=architecture,
        seed=seed,
        configured_epochs=configured_epochs,
        status=status,
        experiment_id=experiment_id,
        checkpoint=experiment_dir / "best_checkpoint.pt",
        run_record=experiment_dir / "run_record.json",
        description=description,
    )

MODEL_SPECS: tuple[ModelSpec, ...] = (
    _spec(
        "final60_baseline_seed42",
        "Final 60 · ResNet34 U-Net · seed 42",
        "baseline",
        "resnet34_unet",
        42,
        60,
        "CONTROLLED",
        "final60_baseline_seed42",
    ),

```

```

        "Controlled 60-epoch baseline run; its validation result is model-specific.",
    ),
    _spec(
        "final60_baseline_seed1337",
        "Final 60 · ResNet34 U-Net · seed 1337",
        "baseline",
        "resnet34_unet",
        1337,
        60,
        "CONTROLLED",
        "final60_baseline_seed1337",
        "Controlled 60-epoch baseline run; its validation result is model-specific.",
    ),
    _spec(
        "final60_hybrid_seed42",
        "Final 60 · Hybrid · seed 42",
        "hybrid",
        "hybrid_segmentation",
        42,
        60,
        "CONTROLLED",
        "final60_hybrid_seed42",
        "Controlled 60-epoch hybrid run; its validation result is model-specific.",
    ),
    _spec(
        "final60_hybrid_seed1337",
        "Final 60 · Hybrid · seed 1337",
        "hybrid",
        "hybrid_segmentation",
        1337,
        60,
        "CONTROLLED",
        "final60_hybrid_seed1337",
        "Controlled 60-epoch hybrid run; its validation result is model-specific.",
    ),
    _spec(
        "final100_hybrid_seed42",
        "Final 100 · Hybrid · seed 42 · exploratory",
        "hybrid",

```

```

        "hybrid_segmentation",
        42,
        100,
        "EXPLORATORY",
        "final100_hybrid_seed42",
        "Exploratory continuation resumed from final60_hybrid_seed42; not a matched comparison.",
    ),
)

```

```

MODEL_BY_ID = {spec.model_id: spec for spec in MODEL_SPECS}
MODEL_IDS = tuple(MODEL_BY_ID)

```

```

def get_model_spec(model_id: str) -> ModelSpec:
    try:
        return MODEL_BY_ID[model_id]
    except KeyError as exc:
        raise UnknownModelError(model_id) from exc

```

```

def public_model_infos() -> list[dict[str, Any]]:
    return [spec.public_info() for spec in MODEL_SPECS]

```

REFERENCES

- [1] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in MICCAI, 2015.
- [2] K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” in CVPR, 2016.
- [3] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in ICLR, 2021 (arXiv:2010.11929).
- [4] X. Wang, W. Li, Z. Wu, ... , “CarDD: A New Dataset for Vision-Based Car Damage Detection,” IEEE Transactions on Intelligent Transportation Systems, 2023. DOI 10.1109/TITS.2023.3258480.
- [5] T. Huynh et al., “VehiDE: A Dataset for Vehicle Damage ... ,” in KSE, 2023.
- [6] J. B. Parsløv et al., “CrashCar101: Crash severity and damage localisation ... ,” in WACV, 2024.
- [7] B. Bhogale, S. Mandal, A. Tiwary and A. Somakumar, “CDA-Net: Camera and depth attention network for damage detection in vehicles,” 2023.
- [8] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal Loss for Dense Object Detection,” in ICCV, 2017.
- [9] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in NeurIPS, 2019.
- [10] LangChain & LangGraph documentation, <https://python.langchain.com> and <https://langchain-ai.github.io/langgraph/>.
- [11] FastAPI documentation, <https://fastapi.tiangolo.com>.
- [12] Next.js documentation, <https://nextjs.org/docs>.
- [13] React documentation, <https://react.dev>.
- [14] Playwright documentation, <https://playwright.dev>.
- [15] OpenCV documentation, <https://docs.opencv.org>.
- [16] AutoInspect-X research report (project internal research document), 2026.
- [17] AutoInspect-X decision records ADR 0010 (hybrid model lineage) and ADR 0011 (photo-first scope; cost/repair/questionnaire removed), 2026.